

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Dina Izzati Elfadheya NIM. 2310817120001

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Dina Izzati Elfadheya

NIM : 2310817120001

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR	4
DAFTAR TABEL.....	5
SOAL 1	6
A. Source Code	7
B. Output Program.....	47
C. Pembahasan.....	53
D. Tautan Git.....	87

DAFTAR GAMBAR

Gambar 1 Halaman Home.....	Error! Bookmark not defined.
Gambar 2 Halaman Detail.....	Error! Bookmark not defined.
Gambar 3 Link Netflix	Error! Bookmark not defined.

DAFTAR TABEL

Tabel 1 Source Code Jawaban DramaDao.kt	7
Tabel 2 Source Code Jawaban DramaDatabase.kt	8
Tabel 3 Source Code Jawaban DramaEntity.kt	8
Tabel 4 Source Code Jawaban DramaApiService.kt.....	9
Tabel 5 Source Code Jawaban DramaDto.kt.....	9
Tabel 6 Source Code Jawaban PopularDramasResponse.kt	9
Tabel 7 Source Code Jawaban ApiClient.kt	12
Tabel 8 Source Code Jawaban DramaRepositoryImpl.kt	14
Tabel 9 Source Code Jawaban DomainMappers.kt.....	15
Tabel 10 Source Code Jawaban Drama.kt.....	16
Tabel 11 Source Code Jawaban DramaRepository.kt	16
Tabel 12 Source Code Jawaban GetDramaUsecase.kt.....	16
Tabel 13 Source Code Jawaban DetailFragment.kt	18
Tabel 14 Source Code Jawaban DramaAdapter.kt	19
Tabel 15 Source Code Jawaban DramaListFragment.kt	24
Tabel 16 Source Code Jawaban DramaViewModel.kt.....	25
Tabel 17 Source Code Jawaban DramaViewModelFactory.kt.....	26
Tabel 18 Source Code Jawaban MainActivity.kt	28
Tabel 19 Source Code Jawaban AppPreferences.kt	28
Tabel 20 Source Code Jawaban Resource.kt.....	28
Tabel 21 Source Code Jawaban Activity_main.xml	29
Tabel 22 Source Code Jawaban Fragment_detail.xml	31
Tabel 23 Source Code Jawaban Fragment_drama_list.xml	33
Tabel 24 Source Code Jawaban item_drama.xml	36
Tabel 25 Source Code Jawaban main_menu.xml.....	36
Tabel 26 Source Code Jawaban nav_graph.xml.....	37
Tabel 27 Source Code Jawaban colors.xml.....	40
Tabel 28 Source Code Jawaban themes.xml	42
Tabel 29 Source Code Jawaban themes.xml	44
Tabel 30 Source Code Jawaban build.gradle.kts.....	46
Tabel 31 Source Code Jawaban AndoidManifest.kt	46

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b Gunakan KotlinX Serialization sebagai library JSON.
- c Gunakan library seperti Coil atau Glide untuk image loading.
- d API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- f Gunakan caching strategy pada Room.
- g Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. data/local/dao/DramaDao.kt

```
1 package com.example.connectapi.data.local.dao
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import
8 com.example.connectapi.data.local.entities.Drama
  Entity
9 import kotlinx.coroutines.flow.Flow
10
11 @Dao
12 interface DramaDao {
13     @Insert(onConflict =
14     OnConflictStrategy.REPLACE)
15     suspend fun insertDramas(dramas:
16     List<DramaEntity>)
17
18     @Query("SELECT * FROM dramas")
19     fun getAllDramas(): Flow<List<DramaEntity>>
20
21     @Query("DELETE FROM dramas")
22     suspend fun deleteAllDramas()
23 }
```

Tabel 1 Source Code Jawaban DramaDao.kt

2. data/local/Database/DramaDatabase

```
1 package
  com.example.connectapi.data.local.Database
2 import android.content.Context
3 import androidx.room.Database
4 import androidx.room.Room
5 import androidx.room.RoomDatabase
6 import
7 com.example.connectapi.data.local.dao.DramaDao
8 import
9 com.example.connectapi.data.local.entities.Drama
  Entity
10
11 @Database(entities = [DramaEntity::class],
12 version = 1, exportSchema = false)
13 abstract class DramaDatabase : RoomDatabase() {
14
15     abstract fun dramaDao(): DramaDao
16 }
```

```

15     companion object {
16         @Volatile
17         private var INSTANCE: DramaDatabase? =
18             null
19
20         fun getDatabase(context: Context):
21             DramaDatabase {
22             return INSTANCE ?:
23             synchronized(this) {
24                 val instance =
25                 Room.databaseBuilder(
26                     context.applicationContext,
27                     DramaDatabase::class.java,
28                     "drama_db"
29                 ).build()
30                 INSTANCE = instance
31                 instance
32             }
33         }
34     }
35 }

```

Tabel 2 Source Code Jawaban DramaDatabase.kt

3. data/local/entities/DramaEntity.kt

```

1 package
2 com.example.connectapi.data.local.entities
3
4 import androidx.room.Entity
5 import androidx.room.PrimaryKey
6
7 @Entity(tableName = "dramas")
8 data class DramaEntity(
9     @PrimaryKey val id: Int,
10     val name: String,
11     val overview: String,
12     val posterPath: String?,
13     val firstAirDate: String?
14 )

```

Tabel 3 Source Code Jawaban DramaEntity.kt

4. data/remote/api/DramaApiService.kt

```

1 package
2 com.example.connectapi.data.local.entities
3
4 import androidx.room.Entity
5 import androidx.room.PrimaryKey
6
7 @Entity(tableName = "dramas")

```


7	data class DramaEntity(8 @PrimaryKey val id: Int, 9 val name: String, 10 val overview: String, 11 val posterPath: String?, 12 val firstAirDate: String? 13 14)
---	--

Tabel 4 Source Code Jawaban DramaApiService.kt

5. data/remote/model/DramaDto.kt

1	package com.example.connectapi.data.remote.model
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class DramaDto(8 val id: Int, 9 val name: String, 10 val overview: String, 11 @SerialName("poster_path") 12 val posterPath: String?, 13 @SerialName("first_air_date") 14 val firstAirDate: String? 15)

Tabel 5 Source Code Jawaban DramaDto.kt

6. data/remote/model/PopularDramasResponse.kt

1	package com.example.connectapi.data.remote.model
2	import kotlinx.serialization.SerialName
3	import kotlinx.serialization.Serializable
4	
5	@Serializable
6	data class PopularDramasResponse(7 @SerialName("page") 8 val page: Int, 9 @SerialName("results") 10 val results: List<DramaDto>, 11 @SerialName("total_pages") 12 val totalPages: Int, 13 @SerialName("total_results") 14 val totalResults: Int 15)

Tabel 6 Source Code Jawaban PopularDramasResponse.kt

7. data/remote/ApiClient.kt

```
1 package com.example.connectapi.data.remote
2
3 import android.util.Log
4 import com.example.connectapi.BuildConfig
5 import
6     com.example.connectapi.data.remote.api.DramaApiService
7 import
8     com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
9 import kotlinx.serialization.json.Json
10 import okhttp3.MediaType.Companion.toMediaType
11 import okhttp3.OkHttpClient
12 import okhttp3.logging.HttpLoggingInterceptor
13 import retrofit2.Retrofit
14 import java.util.concurrent.TimeUnit
15
16 object ApiClient {
17     private const val BASE_URL =
18         "https://api.themoviedb.org/3/"
19     private const val TAG = "ApiClient"
20
21     private val json = Json {
22         ignoreUnknownKeys = true
23         prettyPrint = true
24         isLenient = true
25         coerceInputValues = true
26     }
27
28     private val okHttpClient: OkHttpClient by
29     lazy {
30         val builder = OkHttpClient.Builder()
31         .connectTimeout(30,
32             TimeUnit.SECONDS)
33         .readTimeout(30, TimeUnit.SECONDS)
34         .writeTimeout(30, TimeUnit.SECONDS)
35
36         try {
37             val loggingInterceptor =
38                 HttpLoggingInterceptor().apply {
39                     setLevel(HttpLoggingInterceptor.Level.BODY)
40                 }
41             builder.addInterceptor(loggingInterceptor)
42         } catch (e: NoClassDefFoundError) {
43             Log.e(TAG, "NoClassDefFoundError: " + e.message)
44         }
45     }
46 }
```

```

37         Log.e(TAG, "HttpLoggingInterceptor
class not found. This might be due to missing
dependency.", e)
38     } catch (e: Exception) {
39         Log.e(TAG, "Error initializing
HttpLoggingInterceptor: ${e.message}", e)
40     }
41
42     builder.build()
43 }
44
45     val dramaApiService: DramaApiService by lazy
{
46         val apiKey: String
47         try {
48             apiKey = BuildConfig.TMDB_API_KEY
49             Log.d(TAG, "DEBUG: TMDB_API_KEY yang
DIBACA dari BuildConfig: '$apiKey'")
50
51             if (apiKey.isNullOrEmpty() || apiKey
== "null" || apiKey.contains("null", ignoreCase
= true)) {
52                 Log.e(TAG, "FATAL ERROR:
TMDB_API_KEY masih kosong atau 'null' saat
runtime. Nilai yang dibaca: '$apiKey'")
53                 throw
IllegalStateException("TMDB_API_KEY tidak
ditemukan atau tidak valid di BuildConfig. Nilai
yang dibaca: '$apiKey'")
54             }
55             Retrofit.Builder()
56                 .baseUrl(BASE_URL)
57                 .client(okHttpClient)
58
59                 .addConverterFactory(json.asConverterFactory("ap
plication/json".toMediaType()))
60                 .build()
61             .create(DramaApiService::class.java)
62         } catch (e: IllegalStateException) {
63             Log.e(TAG, "Error in ApiClient
initialization due to invalid API Key:
${e.message}", e)
64             throw e
65         } catch (e: Exception) {
66             Log.e(TAG, "Error initializing
DramaApiService unexpectedly: ${e.message}", e)
67             throw RuntimeException("Gagal
menginisialisasi layanan API. Detail:
${e.localizedMessage}", e)

```

67	}
68	}
69	}

Tabel 7 Source Code Jawaban ApiClient.kt

8. data/remote/repository/DramaRepositoryImpl.kt

```

1  package com.example.connectapi.data.repository
2
3  import android.util.Log
4  import
5  com.example.connectapi.data.local.dao.DramaDao
6  import
7  com.example.connectapi.data.local.entities.Drama
8  Entity
9  import
10 com.example.connectapi.data.remote.api.DramaApiS
11 ervice
12 import com.example.connectapi.domain.model.Drama
13 import
14 com.example.connectapi.domain.repository.DramaRe
15 pository
16 import com.example.connectapi.utils.Resource
17 import
18 com.example.connectapi.utils.toDomainModel
19 import com.example.connectapi.utils.toEntity
20 import kotlinx.coroutines.Dispatchers
21 import kotlinx.coroutines.flow.Flow
22 import kotlinx.coroutines.flow.firstOrNull
23 import kotlinx.coroutines.flow.flow
24 import kotlinx.coroutines.flow.map
25 import kotlinx.coroutines.withContext
26 import retrofit2.HttpException
27 import java.io.IOException
28
29 class DramaRepositoryImpl(
30     private val dramaDao: DramaDao,
31     private val dramaApiService:
32     DramaApiService,
33     private val apiKey: String
34 ) : DramaRepository {
35
36     override fun getPopularDramas():
37     Flow<Resource<List<Drama>>> = flow {
38         emit(Resource.Loading())
39
40         val cachedDramasFlow =
41         dramaDao.getAllDramas().map { entities ->
42             entities.map { it.toDomainModel() }
43         }
44     }

```

```

32         val cachedDramas =
33         cachedDramasFlow.firstOrNull()

34         if (!cachedDramas.isNullOrEmpty()) {
35             emit(Resource.Success(cachedDramas))
36             Log.d("DramaRepositoryImpl",
37 "Berhasil mengambil data dari database lokal
(cached).")
38         } else {
39             Log.d("DramaRepositoryImpl",
40 "Database lokal kosong atau tidak ada cached
data.")
41         }
42         try {
43             val response =
44             dramaApiService.getPopularDramas(apiKey =
apiKey)
45             if (response.isSuccessful) {
46                 val dramaDtos =
47                 response.body()?.results ?: emptyList()
48                 val domainDramas = dramaDtos.map
49                 { it.toDomainModel() }

50                 withContext(Dispatchers.IO) {
51                     dramaDao.deleteAllDramas()
52                     dramaDao.insertDramas(dramaDtos.map {
53                         it.toEntity() })
54                 }
55                 Log.d("DramaRepositoryImpl",
56 "Berhasil mengambil data dari API dan
memperbarui database.")
57                 emit(Resource.Success(domainDramas))
58             } else {
59                 val errorMessage = "API Error:
60                 ${response.code()} ${response.message()}"
61                 Log.e("DramaRepositoryImpl",
62                 errorMessage)
63                 emit(Resource.Error(errorMessage, cachedDramas))
64             }
65         } catch (e: HttpException) {
66             val errorMessage = "Network Error
67             (HTTP ${e.code()}) : ${e.message()}"
68             Log.e("DramaRepositoryImpl",
69             errorMessage, e)
70             emit(Resource.Error(errorMessage,
cachedDramas))
71         } catch (e: IOException) {
72             val errorMessage = "No Internet
73             Connection or API Timeout: ${e.message}"

```

```

63         Log.e("DramaRepositoryImpl",
errorMessage, e)
64         emit(Resource.Error(errorMessage,
cachedDramas))
65     } catch (e: Exception) {
        val errorMessage = "An unexpected
66 error occurred: ${e.localizedMessage}"
67         Log.e("DramaRepositoryImpl",
errorMessage, e)
68         emit(Resource.Error(errorMessage,
cachedDramas))
69     }
    }
70     suspend fun insertDramas(dramas:
71 List<Drama>) {
72         withContext(Dispatchers.IO) {
            dramaDao.insertDramas(dramas.map {
73 it.toEntity() })
74         Log.d("DramaRepositoryImpl", "Data
drama berhasil dimasukkan ke database.")
75     }
    }
76     suspend fun clearAllDramas() {
77         withContext(Dispatchers.IO) {
            dramaDao.deleteAllDramas()
78         Log.d("DramaRepositoryImpl", "Semua
80 data drama di database telah dihapus.")
81     }
    }
82 }
83
84

```

Tabel 8 Source Code Jawaban DramaRepositoryImpl.kt

9. data/remote/utils/DomainMappers.kt

```

1 package com.example.connectapi.utils
2
3 import
com.example.connectapi.data.local.entities.Drama
Entity
4 import
com.example.connectapi.data.remote.model.DramaDt
o
5 import com.example.connectapi.domain.model.Drama
6
7 fun DramaDto.toDomainModel(): Drama {
8     return Drama(
9         id = id,
10        name = name,

```

```

11         overview = overview,
12         posterPath = posterPath,
13         firstAirDate = firstAirDate
14     )
15 }
16 fun DramaEntity.toDomainModel(): Drama {
17     return Drama(
18         id = id,
19         name = name,
20         overview = overview,
21         posterPath = posterPath,
22         firstAirDate = firstAirDate
23     )
24 }
25 fun DramaDto.toEntity(): DramaEntity {
26     return DramaEntity(
27         id = id,
28         name = name,
29         overview = overview,
30         posterPath = posterPath,
31         firstAirDate = firstAirDate
32     )
33 }
34 fun Drama.toEntity(): DramaEntity {
35     return DramaEntity(
36         id = id,
37         name = name,
38         overview = overview,
39         posterPath = posterPath,
40         firstAirDate = firstAirDate
41     )
42 }

```

Tabel 9 Source Code Jawaban DomainMappers.kt

10. domain/model/Drama.kt

```
1 package com.example.connectapi.domain.model
2
3 import android.os.Parcelable
4 import kotlinx.parcelize.Parcelize
5
6 @Parcelize
7 data class Drama(
8     val id: Int,
9     val name: String,
10    val overview: String,
11    val posterPath: String?,
12    val firstAirDate: String?
13 ) : Parcelable
```

Tabel 10 Source Code Jawaban Drama.kt

11. domain/repository/DramaRepository.kt

```
1 package com.example.connectapi.domain.repository
2
3 import com.example.connectapi.domain.model.Drama
4 import com.example.connectapi.utils.Resource
5 import kotlinx.coroutines.flow.Flow
6
7 interface DramaRepository {
8     fun getPopularDramas():
9     Flow<Resource<List<Drama>>>
10 }
11 }
```

Tabel 11 Source Code Jawaban DramaRepository.kt

12. domain/usecase/GetDramaUseCase

```
1 package com.example.connectapi.domain.usecase
2
3 import com.example.connectapi.domain.model.Drama
4 import
5 com.example.connectapi.domain.repository.DramaRe
6 pository
7 import com.example.connectapi.utils.Resource
8 import kotlinx.coroutines.flow.Flow
9
10 class GetDramaUseCase(private val
11 dramaRepository: DramaRepository) {
12     operator fun invoke():
13     Flow<Resource<List<Drama>>> {
14         return
15         dramaRepository.getPopularDramas()
16     }
17 }
```

Tabel 12 Source Code Jawaban GetDramaUseCase.kt

13. presentation/ui/DetailFragment

```
1 package com.example.connectapi.presentation.ui
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import coil.load
9 import
10 com.example.connectapi.databinding.FragmentDetailBinding
11 import com.example.connectapi.domain.model.Drama
12
13 class DetailFragment : Fragment() {
14     private var _binding: FragmentDetailBinding?
15     = null
16     private val binding get() = _binding!!
17
18     override fun onCreateView(
19         inflater: LayoutInflater, container:
20         ViewGroup?,
21         savedInstanceState: Bundle?
22     ): View {
23         _binding =
24         FragmentDetailBinding.inflate(inflater,
25         container, false)
26         return binding.root
27     }
28     override fun onViewCreated(view: View,
29         savedInstanceState: Bundle?) {
30         super.onViewCreated(view,
31         savedInstanceState)
32
33         val drama =
34         arguments?.getParcelable<Drama>("drama")
35         drama?.let {
36             binding.tvDetailTitle.text = it.name
37             binding.tvDetailOverview.text =
38             it.overview
39             binding.tvDetailReleaseDate.text =
40             it.firstAirDate
41             binding.ivDetailPoster.load("https://image.tmdb.
42             org/t/p/w500${it.posterPath}") {
43                 crossfade(true)
44             }
45         }
46     }
47 }
```

34	override fun onDestroyView() {
35	super.onDestroyView()
36	_binding = null
37	}
38	}
39	
40	

Tabel 13 Source Code Jawaban DetailFragment.kt

14. presentation/ui/DramaAdapter.kt

1	package com.example.connectapi.presentation.ui
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.DiffUtil
6	import androidx.recyclerview.widget.ListAdapter
7	import androidx.recyclerview.widget.RecyclerView
8	import coil.load
9	import
	com.example.connectapi.databinding.ItemDramaBinding
10	import com.example.connectapi.domain.model.Drama
11	
12	class DramaAdapter : ListAdapter<Drama,
	DramaAdapter.DramaViewHolder>(DramaDiffCallback(
	)) {
13	
14	var onDetailClickListener: ((Drama) ->
	Unit)? = null
15	var onNetflixClickListener: ((Drama) ->
	Unit)? = null
16	
17	override fun onCreateViewHolder(parent:
	ViewGroup, viewType: Int): DramaViewHolder {
18	val binding =
	ItemDramaBinding.inflate(LayoutInflater.from(par
	ent.context), parent, false)
19	return DramaViewHolder(binding)
20	}
21	
22	override fun onBindViewHolder(holder:
	DramaViewHolder, position: Int) {
23	val drama = getItem(position)
24	holder.bind(drama)
25	}
26	
27	inner class DramaViewHolder(private val
	binding: ItemDramaBinding) :
	RecyclerView.ViewHolder(binding.root) {

28	fun bind(drama: Drama) {
29	binding.tvTitle.text = drama.name
30	binding.tvOverview.text =
	drama.overview
31	binding.tvReleaseDate.text =
	drama.firstAirDate
32	
	binding.ivPoster.load("https://image.tmdb.org/t/
	p/w500\${drama.posterPath}") {
33	crossfade(true)
	}
34	
35	binding.btnDetail.setOnClickListener
36	{
37	onDetailClickListener?.invoke(drama)
	}
38	}
39	}
40	
41	class DramaDiffCallback :
42	DiffUtil.ItemCallback<Drama>() {
43	override fun areItemsTheSame(oldItem:
	Drama, newItem: Drama): Boolean {
44	return oldItem.id == newItem.id
	}
45	
46	override fun areContentsTheSame(oldItem:
47	Drama, newItem: Drama): Boolean {
48	return oldItem == newItem
	}
49	}
50	}
51	
52	

Tabel 14 Source Code Jawaban DramaAdapter.kt

15. presentation/ui/DramaListFragment.kt

1	package com.example.connectapi.presentation.ui
2	
3	import android.os.Bundle
4	import android.util.Log
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import android.widget.Toast
9	import androidx.fragment.app.Fragment
10	import androidx.fragment.app.viewModels
11	import androidx.lifecycle.Lifecycle
12	import androidx.lifecycle.LifecycleScope

```

13 import androidx.lifecycle.repeatOnLifecycle
14 import
15     androidx.navigation.fragment.findNavController
16     import
17     androidx.recyclerview.widget.LinearLayoutManager
18     import
19     com.example.connectapi.BuildConfig
20     import com.example.connectapi.R
21     import
22     com.example.connectapi.data.local.Database.DramaDatabase
23     import
24     com.example.connectapi.data.remote.ApiClient
25     import
26     com.example.connectapi.data.repository.DramaRepositoryImpl
27     import
28     com.example.connectapi.databinding.FragmentDramaListBinding
29     import
30     com.example.connectapi.domain.usecase.GetDramaUseCase
31     import com.example.connectapi.utils.Resource
32     import
33     com.example.connectapi.utils.AppPreferences
34     import
35     androidx.appcompat.app.AppCompatActivity
36     import kotlinx.coroutines.launch
37
38 class DramaListFragment : Fragment() {
39     private var _binding:
40     FragmentDramaListBinding? = null
41     private val binding get() = _binding!!
42
43     private val dramaViewModel: DramaViewModel
44     by viewModels {
45         val dramaDao =
46         DramaDatabase.getDatabase(requireContext()).dramaDao()
47         val dramaApiService =
48         ApiClient.dramaApiService
49         val apiKey = BuildConfig.TMDB_API_KEY
50
51         val dramaRepository =
52         DramaRepositoryImpl(dramaDao, dramaApiService,
53         apiKey)
54         val getDramaUseCase =
55         GetDramaUseCase(dramaRepository)
56     }
57 }

```

```

40 DramaViewModelFactory(requireActivity().applic
41 ation, getDramaUseCase)
42     }
43     private lateinit var dramaAdapter:
DramaAdapter
44     private lateinit var appPreferences:
AppPreferences
45
46     override fun onCreateView(
47         inflater: LayoutInflater, container:
ViewGroup?,
48         savedInstanceState: Bundle?
49     ): View {
50         _binding =
FragmentDramaListBinding.inflate(inflater,
51 container, false)
52         return binding.root
53     }
54     override fun onViewCreated(view: View,
55 savedInstanceState: Bundle?) {
56         super.onViewCreated(view,
57 savedInstanceState)
58
59         appPreferences =
AppPreferences(requireContext())
60
61         setupRecyclerView()
62         observeViewModel()
63
64         val darkModeSwitch =
binding.darkModeSwitch
65
66         darkModeSwitch.isChecked =
appPreferences.getDarkModeState()
67
68         darkModeSwitch.setOnCheckedChangeListener {_,
69 isChecked ->
70 appPreferences.saveDarkModeState(isChecked)
71
72         applyTheme(isChecked)
73     }
74     binding.btnRefreshApi.setOnClickListener {
75         dramaViewModel.refreshDramasFromLocal()
76     }
77     binding.btnClearLocalData.setOnClickListener {
78         clearLocalData()
79     }
80 }

```

```

77     private fun setupRecyclerView() {
78         dramaAdapter = DramaAdapter()
79         binding.rvDrama.apply {
80             layoutManager =
81             LinearLayoutManager(context)
82             adapter = dramaAdapter
83         }
84         dramaAdapter.onDetailClickListener = {
85             drama ->
86                 val bundle = Bundle().apply {
87                     putParcelable("drama", drama)
88                 }
89                 findNavController().navigate(R.id.action_drama_
90                 ListFragment_to_detailFragment, bundle)
91             }
92             dramaAdapter.onNetflixClickListener =
93             { drama ->
94                 Toast.makeText(context, "Tombol
95                 Netflix untuk ${drama.name} diklik!",
96                 Toast.LENGTH_SHORT).show()
97             }
98         }
99         private fun observeViewModel() {
100             viewLifecycleOwner.lifecycleScope.launch {
101                 repeatOnLifecycle(Lifecycle.State.STARTED) {
102                     dramaViewModel.popularDramasState.collect {
103                         resource ->
104                             when (resource) {
105                                 is Resource.Loading ->
106                                 {
107                                     if
108                                     (resource.data.isNullOrEmpty()) {
109                                     binding.progressBar.visibility = View.VISIBLE
110                                     binding.rvDrama.visibility = View.GONE
111                                     binding.tvError.visibility = View.GONE
112                                     } else {
113                                     binding.progressBar.visibility = View.GONE
114                                     binding.rvDrama.visibility = View.VISIBLE
115                                     binding.tvError.visibility = View.GONE
116                                     dramaAdapter.submitList(resource.data)
117                                     }
118                                 }
119                                 is Resource.Success ->
120                                 {
121                                     binding.progressBar.visibility = View.GONE
122                                     binding.rvDrama.visibility = View.VISIBLE
123                                     binding.tvError.visibility = View.GONE

```

```

115                                     resource.data?.let
116 {
117 dramaAdapter.submitList(it)
118                                     }
119                                     }
120                                     is Resource.Error -> {
121 binding.progressBar.visibility = View.GONE
122 binding.rvDrama.visibility = View.GONE
123 binding.tvError.visibility = View.VISIBLE
124 binding.tvError.text = resource.message ?:
    "Terjadi kesalahan tidak diketahui"

125 Toast.makeText(context, "Error:
    ${resource.message}",
126 Toast.LENGTH_LONG).show()
127 Log.e("DramaListFragment", "Error:
    ${resource.message}")
128                                     resource.data?.let
129 {
130                                     if
131 (it.isNotEmpty()) {
132 dramaAdapter.submitList(it)
133 binding.rvDrama.visibility = View.VISIBLE
134 binding.tvError.visibility = View.GONE
135                                     }
136                                     }
137                                     }
138                                     }
139                                     }
140 private fun clearLocalData() {
141     lifecycleScope.launch {
142         try {
143             dramaViewModel.clearLocalData()
144             Toast.makeText(context, "Data
                lokal berhasil dihapus!",
145             Toast.LENGTH_SHORT).show()
146             } catch (e: Exception) {
147                 Toast.makeText(context, "Gagal
                menghapus data lokal: ${e.localizedMessage}",
148                 Toast.LENGTH_SHORT).show()
149                 Log.e("DramaListFragment",
                "Error clearing local data:
                ${e.localizedMessage}", e)
150             }
151         }
152     }
153 }

```

150	private fun applyTheme(isDarkMode: Boolean) {
151	if (isDarkMode) {
152	AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_YES)
153	} else {
154	AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_NO)
155	}
156	requireActivity().recreate()
157	}
158	override fun onDestroyView() {
159	super.onDestroyView()
160	_binding = null
	}
	}

Tabel 15 Source Code Jawaban DramaListFragment.kt

16. presentation/ui/DramaViewModel.kt

1	package com.example.connectapi.presentation.ui
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.connectapi.domain.model.Drama
6	import
	com.example.connectapi.domain.repository.DramaRepository
7	import
	com.example.connectapi.domain.usecase.GetDramaUseCase
8	import com.example.connectapi.utils.Resource
9	import kotlinx.coroutines.flow.MutableStateFlow
10	import kotlinx.coroutines.flow.StateFlow
11	import kotlinx.coroutines.flow.asStateFlow
12	import kotlinx.coroutines.launch
13	
14	class DramaViewModel(
15	private val getDramaUseCase: GetDramaUseCase,
16	private val dramaRepository: DramaRepository
17	) : ViewModel() {
18	
19	private val _popularDramasState =
	MutableStateFlow<Resource<List<Drama>>>(Resource.Loading())
20	

	<pre> val popularDramasState: 21 StateFlow<Resource<List<Drama>>> = 22 _popularDramasState.asStateFlow() 23 24 init { 25 fetchPopularDramas() 26 } 27 private fun fetchPopularDramas() { 28 viewModelScope.launch { 29 getDramaUseCase().collect { resource -> 30 _popularDramasState.value = 31 resource 32 } 33 } 34 fun refreshDramasFromLocal() { 35 fetchPopularDramas() 36 } 37 38 fun clearLocalData() { 39 viewModelScope.launch { 40 (dramaRepository as? com.example.connectapi.data.repository.DramaRepo sitoryImpl)?.clearAllDramas() 41 fetchPopularDramas() 42 } 43 } 44 } </pre>
--	--

Tabel 16 Source Code Jawaban DramaViewModel.kt

17. presentation/ui/DramaViewModelFactory.kt

1	package com.example.connectapi.presentation.ui
2	
3	import android.app.Application
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelProvider
6	import
	com.example.connectapi.data.local.Database.Drama
	Database
7	import
	com.example.connectapi.data.remote.ApiClient
8	import
	com.example.connectapi.data.repository.DramaRepo
	sitoryImpl
9	import
	com.example.connectapi.domain.usecase.GetDramaUs
	eCase
10	import com.example.connectapi.BuildConfig

11	
12	class DramaViewModelFactory(
13	private val application: Application,
14	private val getPopularDramaUseCase:
	GetDramaUseCase
15	) : ViewModelProvider.Factory {
16	override fun <T : ViewModel>
	create(modelClass: Class<T>): T {
17	if
	(modelClass.isAssignableFrom(DramaViewModel::cla
	ss.java)) {
18	
19	val dramaDao =
	DramaDatabase.getDatabase(application).dramaDao(
	)
20	val dramaApiService =
	ApiClient.dramaApiService
21	val apiKey =
	BuildConfig.TMDB_API_KEY
22	val dramaRepository =
	DramaRepositoryImpl(dramaDao, dramaApiService,
	apiKey)
23	val getDramaUseCase =
	GetDramaUseCase(dramaRepository)
24	
25	@Suppress("UNCHECKED_CAST")
26	return
	DramaViewModel(getDramaUseCase, dramaRepository)
	as T
27	}
28	throw IllegalArgumentException("Unknown
	ViewModel class")
29	}
30	}

Tabel 17 Source Code Jawaban DramaViewModelFactory.kt

18. MainActivity.kt

1	package com.example.connectapi.presentation.ui
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import androidx.appcompat.app.AppCompatActivityDelegate
6	import
	androidx.navigation.fragment.NavHostFragment
7	import
	androidx.navigation.ui.setupActionBarWithNavController
8	roller

```

import
9  com.example.connectapi.databinding.ActivityMainB
10 inding
11 import
12 com.example.connectapi.utils.AppPreferences
13
14 class MainActivity : AppCompatActivity() {
15     private lateinit var binding:
ActivityMainBinding
16     private lateinit var appPreferences:
AppPreferences
17     override fun onCreate(savedInstanceState:
Bundle?) {
18         super.onCreate(savedInstanceState)
19         binding =
ActivityMainBinding.inflate(layoutInflater)
20         setContentView(binding.root)
21
22         appPreferences = AppPreferences(this)
23
24         applySavedTheme()
25
26         setSupportActionBar(binding.toolbar)
27
28         val navHostFragment =
supportFragmentManager
29         .findFragmentById(binding.navHostFragment.id) as
NavHostFragment
30         val navController =
navHostFragment.navController
31         setupActionBarWithNavController(navController)
32     }
33     override fun onSupportNavigateUp(): Boolean
{
34         val navHostFragment =
supportFragmentManager
35         .findFragmentById(binding.navHostFragment.id) as
NavHostFragment
36         val navController =
navHostFragment.navController
37         return navController.navigateUp() ||
super.onSupportNavigateUp()
38     }
39     private fun applySavedTheme() {
40         val isDarkMode =
appPreferences.getDarkModeState()
41         AppCompatDelegate.setDefaultNightMode(
if (isDarkMode)
42         AppCompatDelegate.MODE_NIGHT_YES

```

43	else AppCompatActivity.MODE_NIGHT_NO
44	)
	}
	}

Tabel 18 Source Code Jawaban MainActivity.kt

19. Utils/AppPreferences.kt

1	package com.example.connectapi.utils
2	
3	import android.content.Context
4	import android.content.SharedPreferences
5	
6	class AppPreferences(context: Context) {
7	private val PREFS_NAME = "app_prefs"
8	private val KEY_DARK_MODE = "dark_mode"
9	private val sharedPrefs: SharedPreferences
10	
11	init {
12	sharedPrefs =
	context.getSharedPreferences(PREFS_NAME,
	Context.MODE_PRIVATE)
13	}
14	fun saveDarkModeState(isDarkMode: Boolean) {
15	sharedPrefs.edit().putBoolean(KEY_DARK_MODE,
	isDarkMode).apply()
16	}
17	fun getDarkModeState(): Boolean {
18	return
	sharedPrefs.getBoolean(KEY_DARK_MODE, false)
19	}
20	}

Tabel 19 Source Code Jawaban AppPreferences.kt

20. Utils/Resource.kt

1	package com.example.connectapi.utils
2	
3	sealed class Resource<T>(val data: T? = null,
	val message: String? = null) {
4	class Loading<T>(data: T? = null) :
	Resource<T>(data)
5	class Success<T>(data: T) :
	Resource<T>(data)
6	class Error<T>(message: String, data: T? =
	null) : Resource<T>(data, message)
7	}

Tabel 20 Source Code Jawaban Resource.kt

21. Activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".presentation.ui.MainActivity">
8	
9	<com.google.android.material.appbar.AppBarLayout
10	android:id="@+id/app_bar_layout"
11	android:layout_width="match_parent"
12	android:layout_height="wrap_content"
13	app:layout_constraintTop_toTopOf="parent">
14	
15	<androidx.appcompat.widget.Toolbar
16	android:id="@+id/toolbar"
17	android:layout_width="match_parent"
18	android:layout_height="53dp"
19	android:background="?attr/colorPrimary"
20	app:titleTextColor="@android:color/white" />
21	</com.google.android.material.appbar.AppBarLayout>
22	<androidx.fragment.app.FragmentContainerView
23	android:id="@+id/nav_host_fragment"
24	android:name="androidx.navigation.fragment.NavHostFragment"
25	android:layout_width="0dp"
26	android:layout_height="0dp"
27	app:defaultNavHost="true"
28	app:layout_constraintBottom_toBottomOf="parent"
29	app:layout_constraintEnd_toEndOf="parent"
30	app:layout_constraintStart_toStartOf="parent"
31	app:layout_constraintTop_toBottomOf="@id/app_bar_layout"
32	app:navGraph="@navigation/nav_graph" />
33	
34	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 21 Source Code Jawaban Activity_main.xml

22. fragment_detail.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLa
   layout
3  xmlns:android="http://schemas.android.com/apk/res/android"
4  xmlns:app="http://schemas.android.com/apk/res-auto"
5  xmlns:tools="http://schemas.android.com/tools"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8  tools:context=".presentation.ui.DetailFragment"
   ">
9
10     <ImageView
11         android:id="@+id/iv_detail_poster"
12         android:layout_width="match_parent"
13         android:layout_height="250dp"
14         android:scaleType="centerCrop"
15     android:src="@drawable/ic_launcher_background"
16     app:layout_constraintTop_toTopOf="parent"
17     app:layout_constraintStart_toStartOf="parent"
18     app:layout_constraintEnd_toEndOf="parent" />
19
20     <TextView
21         android:id="@+id/tv_detail_title"
22         android:layout_width="0dp"
23         android:layout_height="wrap_content"
24         android:layout_marginTop="16dp"
25         android:layout_marginStart="16dp"
26         android:layout_marginEnd="16dp"
27         android:text="Judul Drama"
28         android:textSize="24sp"
29         android:textStyle="bold"
30     app:layout_constraintTop_toBottomOf="@id/iv_detail_poster"
31     app:layout_constraintStart_toStartOf="parent"
32     app:layout_constraintEnd_toEndOf="parent" />
33
34     <TextView
35     android:id="@+id/tv_detail_release_date"
36         android:layout_width="0dp"
37         android:layout_height="wrap_content"
38         android:layout_marginTop="8dp"
39         android:layout_marginStart="16dp"
40         android:layout_marginEnd="16dp"
41         android:text="Tanggal Rilis: YYYY-MM-DD"
```

42	android:textSize="16sp"
43	android:alpha="0.7"
44	app:layout_constraintTop_toBottomOf="@id/tv_detail_title"
45	app:layout_constraintStart_toStartOf="parent"
46	app:layout_constraintEnd_toEndOf="parent" />
47	
48	<TextView
49	android:id="@+id/tv_detail_overview"
50	android:layout_width="0dp"
51	android:layout_height="0dp"
52	android:layout_marginTop="16dp"
53	android:layout_marginStart="16dp"
54	android:layout_marginEnd="16dp"
55	android:layout_marginBottom="16dp"
56	android:text="Deskripsi panjang drama..."
57	android:textSize="14sp"
58	android:scrollbars="vertical"
59	app:layout_constraintTop_toBottomOf="@id/tv_detail_release_date"
60	app:layout_constraintBottom_toBottomOf="parent"
61	"
62	app:layout_constraintStart_toStartOf="parent"
63	app:layout_constraintEnd_toEndOf="parent" />
64	</androidx.constraintlayout.widget.ConstraintLayout>

Tabel 22 Source Code Jawaban Fragment_detail.xml

23. fragment_drama_list.xml

1	<androidx.constraintlayout.widget.ConstraintLayout
2	xmlns:android="http://schemas.android.com/apk/res/android"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	tools:context=".presentation.ui.DramaListFragment">
8	
9	<Button
10	android:id="@+id/btn_refresh_api"
11	android:layout_width="0dp"
12	android:layout_height="wrap_content"
13	android:text="Refresh dari API"
14	app:layout_constraintTop_toTopOf="parent"
15	app:layout_constraintStart_toStartOf="parent"

```

16 app:layout_constraintEnd_toStartOf="@id/guidelin
   e_center"
17         android:layout_marginTop="16dp"
18         android:layout_marginStart="16dp"
19         android:layout_marginEnd="8dp" />
20
21     <Button
22         android:id="@+id/btn_clear_local_data"
23         android:layout_width="0dp"
24         android:layout_height="wrap_content"
25         android:text="Hapus Data Lokal"
26 app:layout_constraintTop_toTopOf="@id/btn_refres
   h_api"
27 app:layout_constraintStart_toEndOf="@id/guidelin
   e_center"
28         app:layout_constraintEnd_toEndOf="parent"
29         android:layout_marginStart="8dp"
30         android:layout_marginEnd="16dp" />
31
32     <androidx.constraintlayout.widget.Guideline
33         android:id="@+id/guideline_center"
34         android:layout_width="wrap_content"
35         android:layout_height="wrap_content"
36         android:orientation="vertical"
37         app:layout_constraintGuide_percent="0.5"
38     />
39 <com.google.android.material.switchmaterial.Swit
   chMaterial
40         android:id="@+id/darkModeSwitch"
41         android:layout_width="wrap_content"
42         android:layout_height="wrap_content"
43         android:layout_marginTop="8dp"
44         android:padding="8dp"
45         android:text="Aktifkan Dark Mode"
46 android:textColor="?attr/colorOnBackground"
47         android:textSize="16sp"
48         app:layout_constraintEnd_toEndOf="parent"
49 app:layout_constraintHorizontal_bias="0.862"
50 app:layout_constraintStart_toStartOf="parent"
51 app:layout_constraintTop_toBottomOf="@id/btn_cle
   ar_local_data" />
52
53     <ProgressBar
54         android:id="@+id/progressBar"
55         android:layout_width="wrap_content"
56         android:layout_height="wrap_content"
57         android:visibility="gone"
58 android:indeterminateTint="?attr/colorPrimary"

```


59	app:layout_constraintBottom_toBottomOf="parent"
60	app:layout_constraintEnd_toEndOf="parent"
61	app:layout_constraintStart_toStartOf="parent"
62	app:layout_constraintTop_toBottomOf="@id/darkMod eSwitch"
63	tools:visibility="visible" />
64	
65	<androidx.recyclerview.widget.RecyclerView
66	android:id="@+id/rvDrama"
67	android:layout_width="0dp"
68	android:layout_height="0dp"
	android:paddingTop="8dp"
	app:layout_constraintBottom_toBottomOf="parent"
69	app:layout_constraintEnd_toEndOf="parent"
70	
71	app:layout_constraintStart_toStartOf="parent"
72	
73	app:layout_constraintTop_toBottomOf="@id/darkMod eSwitch"
74	
75	tools:listitem="@layout/item_drama" />
76	<TextView
77	android:id="@+id/tvError"
78	android:layout_width="wrap_content"
79	android:layout_height="wrap_content"
80	android:text="Terjadi kesalahan!"
81	android:textColor="@color/red_error"
82	android:textSize="16sp"
83	android:visibility="gone"
84	app:layout_constraintBottom_toBottomOf="parent"
85	app:layout_constraintEnd_toEndOf="parent"
86	app:layout_constraintStart_toStartOf="parent"
87	app:layout_constraintTop_toBottomOf="@id/darkMod eSwitch"
88	
89	tools:visibility="visible" />
90	</androidx.constraintlayout.widget.ConstraintLay out>

Tabel 23 Source Code Jawaban Fragment_drama_list.xml

24. item_drama.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/re s/android"
4	xmlns:app="http://schemas.android.com/apk/res-

	auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:layout_width="match_parent"
7	android:layout_height="wrap_content"
8	android:layout_margin="8dp"
9	app:cardCornerRadius="8dp"
10	app:cardElevation="4dp">
11	
12	<androidx.constraintlayout.widget.ConstraintLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:padding="16dp">
16	
17	<ImageView
18	android:id="@+id/iv_poster"
19	android:layout_width="100dp"
20	android:layout_height="150dp"
21	android:scaleType="centerCrop"
22	android:src="@drawable/ic_launcher_background"
23	app:layout_constraintStart_toStartOf="parent"
24	app:layout_constraintTop_toTopOf="parent"
25	app:layout_constraintBottom_toBottomOf="parent"
26	/>
27	<TextView
28	android:id="@+id/tv_title"
29	android:layout_width="0dp"
30	android:layout_height="wrap_content"
31	android:layout_marginStart="16dp"
32	android:layout_marginEnd="8dp"
33	android:text="Judul Drama"
34	android:textSize="18sp"
35	android:textStyle="bold"
36	app:layout_constraintTop_toTopOf="parent"
37	app:layout_constraintStart_toEndOf="@id/iv_poster"
38	app:layout_constraintEnd_toEndOf="parent" />
39	
40	<TextView
41	android:id="@+id/tv_release_date"
42	android:layout_width="0dp"
43	android:layout_height="wrap_content"
44	android:layout_marginStart="16dp"
45	android:layout_marginTop="4dp"
46	android:layout_marginEnd="8dp"
47	android:text="Tanggal Rilis: YYYY-
48	MM-DD"
	android:textSize="14sp"

```

49         android:alpha="0.7"
50 app:layout_constraintTop_toBottomOf="@id/tv_title"
51 app:layout_constraintStart_toEndOf="@id/iv_poster"
52 app:layout_constraintEnd_toEndOf="parent" />
53
54         <TextView
55             android:id="@+id/tv_overview"
56             android:layout_width="0dp"
57             android:layout_height="wrap_content"
58             android:layout_marginStart="16dp"
59             android:layout_marginTop="8dp"
60             android:layout_marginEnd="8dp"
61             android:maxLines="3"
62             android:ellipsize="end"
63             android:text="Ini adalah ringkasan
64 singkat dari drama..."
65             android:textSize="14sp"
66 app:layout_constraintTop_toBottomOf="@id/tv_release_date"
67 app:layout_constraintStart_toEndOf="@id/iv_poster"
68 app:layout_constraintEnd_toEndOf="parent" />
69
70         <LinearLayout
71             android:layout_width="0dp"
72             android:layout_height="wrap_content"
73             android:orientation="horizontal"
74             android:layout_marginTop="8dp"
75 app:layout_constraintTop_toBottomOf="@id/tv_overview"
76 app:layout_constraintStart_toEndOf="@id/iv_poster"
77 app:layout_constraintEnd_toEndOf="parent"
78 app:layout_constraintBottom_toBottomOf="parent"
79             android:layout_marginStart="16dp">
80
81             <Button
82                 android:id="@+id/btn_detail"
83                 android:layout_width="0dp"
84                 android:layout_height="wrap_content"
85                 android:layout_weight="1"
86                 android:text="Detail"
87                 android:layout_marginEnd="4dp"/>
88
89             </LinearLayout>
90

```

91	<pre> </androidx.constraintlayout.widget.ConstraintLayout> </androidx.cardview.widget.CardView> </pre>
----	--

Tabel 24 Source Code Jawaban item_drama.xml

25. main_menu.xml

1	<pre><?xml version="1.0" encoding="utf-8"?></pre>
2	<pre><menu</pre>
	<pre> xmlns:android="http://schemas.android.com/apk/res/android"</pre>
3	<pre> xmlns:app="http://schemas.android.com/apk/res-auto"></pre>
4	<pre> <item</pre>
	<pre> android:id="@+id/action_dark_mode_switch"</pre>
5	<pre> android:title="Dark Mode"</pre>
6	<pre> app:actionViewClass="com.google.android.material</pre>
7	<pre> .switchmaterial.SwitchMaterial"</pre>
8	<pre> app:showAsAction="always" /></pre>
9	<pre></menu></pre>

Tabel 25 Source Code Jawaban main_menu.xml

26. nav_graph.xml

1	<pre><?xml version="1.0" encoding="utf-8"?></pre>
2	<pre><navigation</pre>
	<pre> xmlns:android="http://schemas.android.com/apk/res/android"</pre>
3	<pre> xmlns:app="http://schemas.android.com/apk/res-auto"</pre>
4	<pre> xmlns:tools="http://schemas.android.com/tools"</pre>
5	<pre> android:id="@+id/nav_graph"</pre>
6	<pre> app:startDestination="@id/dramaListFragment"></pre>
7	<pre> <fragment</pre>
8	<pre> android:id="@+id/dramaListFragment"</pre>
9	<pre> android:name="com.example.connectapi.presentation.ui.DramaListFragment"</pre>
10	<pre> android:label="Series Lawas Populer"</pre>
11	<pre> tools:layout="@layout/fragment_drama_list"></pre>
12	<pre> <action</pre>
13	<pre> android:id="@+id/action_dramaListFragment_to_detailFragment"</pre>
14	<pre> app:destination="@id/detailFragment"</pre>
15	<pre> /></pre>
16	<pre> </fragment></pre>
17	<pre> <fragment</pre>
18	<pre> <fragment</pre>

19	android:id="@+id/detailFragment"
	android:name="com.example.connectapi.presentation.ui.DetailFragment"
20	android:label="Drama Detail"
21	tools:layout="@layout/fragment_detail">
22	<argument
23	android:name="drama"
24	app:argType="com.example.connectapi.domain.model.Drama" />
25	</fragment>
26	</navigation>

Tabel 26 Source Code Jawaban nav_graph.xml

27. colors.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<resources>
3	
4	<color name="black">#FF000000</color>
5	<color name="white">#FFFFFFFF</color>
6	<color name="red_error">#FFDA4433</color>
7	
8	<color
	name="md_theme_light_primary">#6750A4</color>
9	<color
	name="md_theme_light_onPrimary">#FFFFFF</color>
10	<color
	name="md_theme_light_primaryContainer">#EADDDFF</color>
11	<color
	name="md_theme_light_onPrimaryContainer">#21005D</color>
12	<color
	name="md_theme_light_secondary">#625B71</color>
13	<color
	name="md_theme_light_onSecondary">#FFFFFF</color>
14	<color
	name="md_theme_light_secondaryContainer">#E8DEF8</color>
15	<color
	name="md_theme_light_onSecondaryContainer">#1D192B</color>
16	<color
	name="md_theme_light_tertiary">#7D5260</color>
17	<color
	name="md_theme_light_onTertiary">#FFFFFF</color>
18	<color
	name="md_theme_light_tertiaryContainer">#FFD8E4</color>

19	<color name="md_theme_light_onTertiaryContainer">#31111D</color> <color
20	name="md_theme_light_error">#BA1A1A</color> <color
21	name="md_theme_light_onError">#FFFFFF</color> <color
22	name="md_theme_light_errorContainer">#FFDAD6</color> <color
23	name="md_theme_light_onErrorContainer">#410002</color> <color
24	name="md_theme_light_background">#FFFBFE</color> <color
25	name="md_theme_light_onBackground">#1C1B1F</color> <color
26	name="md_theme_light_surface">#FFFBFE</color> <color
27	name="md_theme_light_onSurface">#1C1B1F</color> <color
28	name="md_theme_light_surfaceVariant">#E7E0EC</color> <color
29	name="md_theme_light_onSurfaceVariant">#49454F</color> <color
30	name="md_theme_light_outline">#7A757F</color> <color
31	name="md_theme_light_shadow">#000000</color> <color
32	name="md_theme_light_inverseSurface">#313034</color> <color
33	name="md_theme_light_inverseOnSurface">#F4EFF4</color> <color
34	name="md_theme_light_inversePrimary">#D0BCFF</color> <color
35	name="md_theme_light_surfaceTint">#6750A4</color> >
36	<color
37	name="md_theme_dark_primary">#D0BCFF</color> <color
38	name="md_theme_dark_onPrimary">#381E72</color>

39	<pre> <color name="md_theme_dark_primaryContainer">#4F378B</c olor> <color 40 name="md_theme_dark_onPrimaryContainer">#EADDDFF< /color> <color 41 name="md_theme_dark_secondary">#CCC2DC</color> <color 42 name="md_theme_dark_onSecondary">#332D41</color> <color 43 name="md_theme_dark_secondaryContainer">#4A4458< /color> <color 44 name="md_theme_dark_onSecondaryContainer">#E8DEF 8</color> <color 45 name="md_theme_dark_tertiary">#EFB8C8</color> <color 46 name="md_theme_dark_onTertiary">#492532</color> <color 47 name="md_theme_dark_tertiaryContainer">#633B48</ color> <color 48 name="md_theme_dark_onTertiaryContainer">#FFD8E4 </color> <color 49 name="md_theme_dark_error">#FFB4AB</color> <color 50 name="md_theme_dark_onError">#690005</color> <color 51 name="md_theme_dark_errorContainer">#93000A</col or> <color 52 name="md_theme_dark_onErrorContainer">#FFDAD6</c olor> <color 53 name="md_theme_dark_background">#1C1B1F</color> <color 54 name="md_theme_dark_onBackground">#E6E1E5</color > <color 55 name="md_theme_dark_surface">#1C1B1F</color> <color 56 name="md_theme_dark_onSurface">#E6E1E5</color> <color 57 name="md_theme_dark_surfaceVariant">#49454F</col or> </pre>
----	--

58	<code><color</code>
	<code>name="md_theme_dark_onSurfaceVariant">#CCC4D4</color></code>
	<code><color</code>
59	<code>name="md_theme_dark_outline">#948F99</color></code>
	<code><color</code>
60	<code>name="md_theme_dark_shadow">#000000</color></code>
	<code><color</code>
61	<code>name="md_theme_dark_inverseSurface">#E6E1E5</color></code>
	<code><color</code>
62	<code>name="md_theme_dark_inverseOnSurface">#1C1B1F</color></code>
	<code><color</code>
63	<code>name="md_theme_dark_inversePrimary">#6750A4</color></code>
	<code><color</code>
64	<code>name="md_theme_dark_surfaceTint">#D0BCFF</color></code>
	<code></resources></code>
65	

Tabel 27 Source Code Jawaban colors.xml

28. themes.xml (light)

1	<code><resources</code>
	<code>xmlns:tools="http://schemas.android.com/tools"></code>
2	<code><!-- Base application theme. --></code>
3	<code><style name="Base.Theme.ConnectAPI"</code>
	<code>parent="Theme.Material3.DayNight.NoActionBar"></code>
4	<code><!-- Customize your light theme here. --</code>
	<code>></code>
5	<code><item</code>
	<code>name="colorPrimary">@color/md_theme_light_primary</item></code>
6	<code><item</code>
	<code>name="colorOnPrimary">@color/md_theme_light_onPrimary</item></code>
7	<code><item</code>
	<code>name="colorPrimaryContainer">@color/md_theme_light_primaryContainer</item></code>
8	<code><item</code>
	<code>name="colorOnPrimaryContainer">@color/md_theme_light_onPrimaryContainer</item></code>
9	<code><item</code>
	<code>name="colorSecondary">@color/md_theme_light_secondary</item></code>
10	<code><item</code>
	<code>name="colorOnSecondary">@color/md_theme_light_onSecondary</item></code>
11	<code><item</code>

	name="colorSecondaryContainer">@color/md_theme_l ight_secondaryContainer</item>
12	<item name="colorOnSecondaryContainer">@color/md_theme _light_onSecondaryContainer</item>
13	<item name="colorTertiary">@color/md_theme_light_terti ary</item>
14	<item name="colorOnTertiary">@color/md_theme_light_onT ertiary</item>
15	<item name="colorTertiaryContainer">@color/md_theme_li ght_tertiaryContainer</item>
16	<item name="colorOnTertiaryContainer">@color/md_theme_ light_onTertiaryContainer</item>
17	<item name="colorError">@color/md_theme_light_error</i tem>
18	<item name="colorOnError">@color/md_theme_light_onErro r</item>
19	<item name="colorErrorContainer">@color/md_theme_light _errorContainer</item>
20	<item name="colorOnErrorContainer">@color/md_theme_lig ht_onErrorContainer</item>
21	<item name="android:colorBackground">@color/md_theme_l ight_background</item>
22	<item name="colorOnBackground">@color/md_theme_light_o nBackground</item>
23	<item name="colorSurface">@color/md_theme_light_surfac e</item>
24	<item name="colorOnSurface">@color/md_theme_light_onSu rface</item>
25	<item name="colorSurfaceVariant">@color/md_theme_light _surfaceVariant</item>
26	<item name="colorOnSurfaceVariant">@color/md_theme_lig ht_onSurfaceVariant</item>
27	<item name="colorOutline">@color/md_theme_light_outlin

28	e</item>
	<item
	name="colorPrimaryInverse">@color/md_theme_light
	_inversePrimary</item>
29	<item name="android:statusBarColor"
	tools:targetApi="1">?attr/colorPrimary</item>
30	<item name="android:navigationBarColor"
	tools:targetApi="1">?attr/colorSurface</item>
31	</style>
32	
33	<style name="Theme.ConnectAPI"
	parent="Base.Theme.ConnectAPI" />
34	</resources>

Tabel 28 Source Code Jawaban themes.xml

29. themes.xml (night)

1	<resources
	xmlns:tools="http://schemas.android.com/tools">
2	<!-- Base application theme. -->
3	<style name="Base.Theme.ConnectAPI"
	parent="Theme.Material3.DayNight.NoActionBar">
4	<!-- Customize your dark theme here. -->
5	<item
	name="colorPrimary">@color/md_theme_dark_primary
	</item>
6	<item
	name="colorOnPrimary">@color/md_theme_dark_onPri
	mary</item>
7	<item
	name="colorPrimaryContainer">@color/md_theme_dar
	k_primaryContainer</item>
8	<item
	name="colorOnPrimaryContainer">@color/md_theme_d
	ark_onPrimaryContainer</item>
9	<item
	name="colorSecondary">@color/md_theme_dark_secon
	dary</item>
10	<item
	name="colorOnSecondary">@color/md_theme_dark_onS
	econdary</item>
11	<item
	name="colorSecondaryContainer">@color/md_theme_d
	ark_secondaryContainer</item>
12	<item
	name="colorOnSecondaryContainer">@color/md_theme
	_dark_onSecondaryContainer</item>
13	<item
	name="colorTertiary">@color/md_theme_dark_tertia
	ry</item>

14	<item name="colorOnTertiary">@color/md_theme_dark_onTertiary</item>
15	<item name="colorTertiaryContainer">@color/md_theme_dark_tertiaryContainer</item>
16	<item name="colorOnTertiaryContainer">@color/md_theme_dark_onTertiaryContainer</item>
17	<item name="colorError">@color/md_theme_dark_error</item>
18	<item name="colorOnError">@color/md_theme_dark_onError</item>
19	<item name="colorErrorContainer">@color/md_theme_dark_errorContainer</item>
20	<item name="colorOnErrorContainer">@color/md_theme_dark_onErrorContainer</item>
21	<item name="android:colorBackground">@color/md_theme_dark_background</item>
22	<item name="colorOnBackground">@color/md_theme_dark_onBackground</item>
23	<item name="colorSurface">@color/md_theme_dark_surface</item>
24	<item name="colorOnSurface">@color/md_theme_dark_onSurface</item>
25	<item name="colorSurfaceVariant">@color/md_theme_dark_surfaceVariant</item>
26	<item name="colorOnSurfaceVariant">@color/md_theme_dark_onSurfaceVariant</item>
27	<item name="colorOutline">@color/md_theme_dark_outline</item>
28	<item name="colorPrimaryInverse">@color/md_theme_dark_inversePrimary</item>
29	<item name="android:statusBarColor" tools:targetApi="1">?attr/colorPrimary</item>
30	<item name="android:navigationBarColor" tools:targetApi="1">?attr/colorSurface</item>

31	</style>
32	</resources>

Tabel 29 Source Code Jawaban themes.xml

30. build.gradle.kts (module:app)

1	plugins {
2	alias(libs.plugins.android.application)
3	alias(libs.plugins.kotlin.android)
4	id("kotlin-parcelize")
5	id("org.jetbrains.kotlin.plugin.serialization")
	version "2.0.0"
6	id("kotlin-kapt")
7	}
8	
9	android {
10	namespace = "com.example.connectapi"
11	compileSdk = 35
12	
13	defaultConfig {
14	applicationId = "com.example.connectapi"
15	minSdk = 30
16	targetSdk = 35
17	versionCode = 1
18	versionName = "1.0"
19	testInstrumentationRunner =
	"androidx.test.runner.AndroidJUnitRunner"
20	buildConfigField("String",
	"TMDB_API_KEY",
	"\"c85206126e56c036e856104caa98a28a\"")
21	}
22	
23	buildTypes {
24	release {
25	isMinifyEnabled = false
26	proguardFiles(
27	getDefaultProguardFile("proguard-android-
	optimize.txt"),
28	"proguard-rules.pro"
29	)
30	}
31	}
32	
33	compileOptions {
34	sourceCompatibility =
	JavaVersion.VERSION_11
35	targetCompatibility =
	JavaVersion.VERSION_11
36	}

```

37
38     kotlinOptions {
39         jvmTarget = "11"
40     }
41
42     buildFeatures {
43         viewBinding = true
44         buildConfig = true
45     }
46 }
47
48 dependencies {
49     implementation(libs.androidx.core.ktx)
50     implementation(libs.androidx.appcompat)
51     implementation(libs.material)
52     implementation(libs.androidx.activity)
53     implementation(libs.androidx.constraintlayout)
54     implementation("androidx.navigation:navigation-
fragment-ktx:2.7.7")
55     implementation("androidx.navigation:navigation-
ui-ktx:2.7.7")
56     implementation("io.coil-kt:coil:2.6.0")
57     testImplementation(libs.junit)
58     androidTestImplementation(libs.androidx.junit)
59     androidTestImplementation(libs.androidx.espresso
.core)
60     testImplementation("androidx.room:room-
compiler-processing-testing:2.7.1")
61     implementation("androidx.room:room-
runtime:2.6.1")
62     kapt("androidx.room:room-compiler:2.6.1")
63     implementation("androidx.room:room-
ktx:2.6.1")
64     implementation("com.squareup.retrofit2:retrofit:
2.9.0")
65     implementation("com.squareup.okhttp3:okhttp:4.12
.0")
66     implementation("com.squareup.okhttp3:logging-
interceptor:4.12.0")
67     implementation("org.jetbrains.kotlinx:kotlinx-
serialization-json:1.6.2")
68     implementation("com.jakewharton.retrofit:retrofi
t2-kotlinx-serialization-converter:1.0.0")
69     implementation("org.jetbrains.kotlinx:kotlinx-
coroutines-android:1.7.3")
70     implementation("androidx.lifecycle:lifecycle-
viewmodel-ktx:2.7.0")
71     implementation("androidx.lifecycle:lifecycle-
runtime-ktx:2.7.0")

```

72	implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.7.0")
73	}

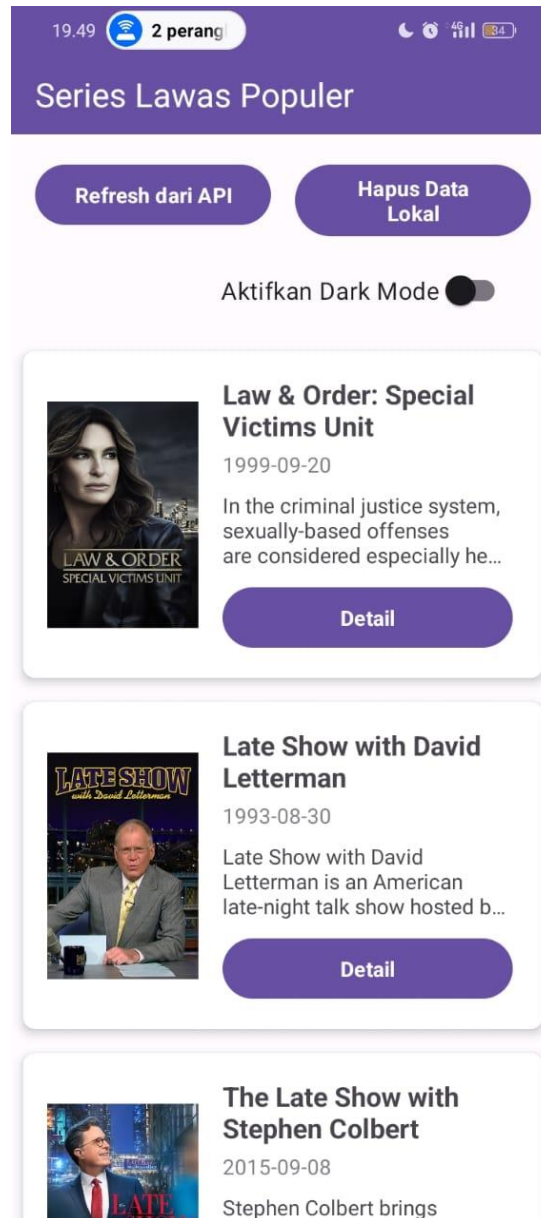
Tabel 30 Source Code Jawaban build.gradle.kts

31. AndroidManifest.kt

1	<?xml version="1.0" encoding="utf-8"?>
2	<manifest
	xmlns:android="http://schemas.android.com/apk/re
	s/android"
3	xmlns:tools="http://schemas.android.com/tools">
4	
5	<uses-permission
	android:name="android.permission.INTERNET" />
6	<application
7	android:allowBackup="true"
8	android:dataExtractionRules="@xml/data_extrac
	n_rules"
9	android:fullBackupContent="@xml/backup_rules"
10	android:icon="@mipmap/ic_launcher"
11	android:label="@string/app_name"
12	android:roundIcon="@mipmap/ic_launcher_round"
13	android:supportsRtl="true"
14	android:theme="@style/Theme.ConnectAPI"
15	tools:targetApi="31">
16	<activity
17	android:name=".presentation.ui.MainActivity"
18	android:exported="true">
19	<intent-filter>
20	<action
	android:name="android.intent.action.MAIN" />
21	<category
	android:name="android.intent.category.LAUNCHER"
	/>
22	</intent-filter>
23	</activity>
24	</application>
25	
26	</manifest>

Tabel 31 Source Code Jawaban AndoidManifest.kt

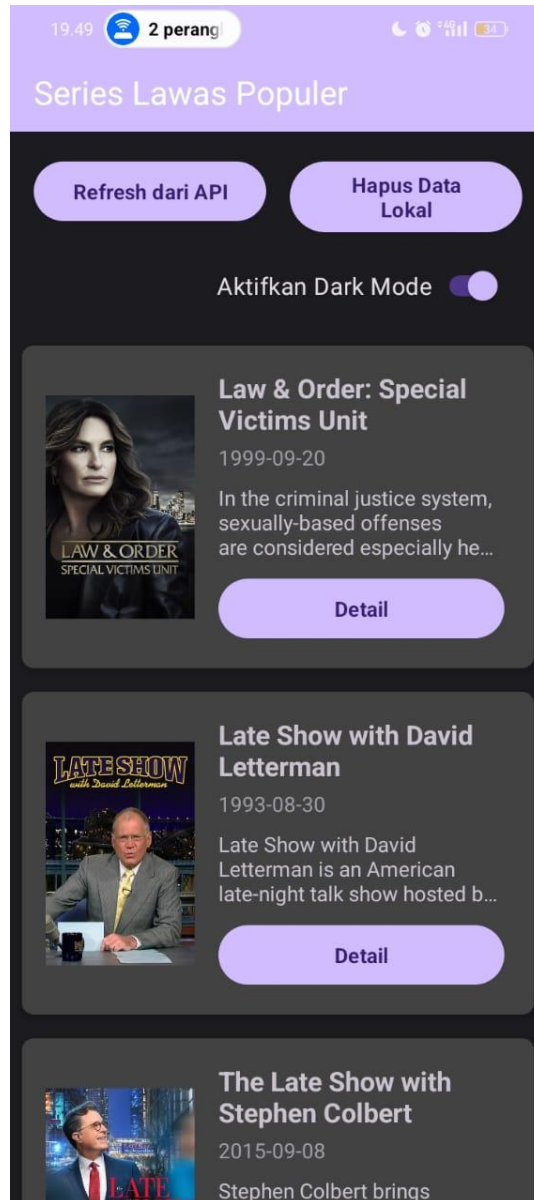
B. Output Program



Gambar 1 Halaman Utama



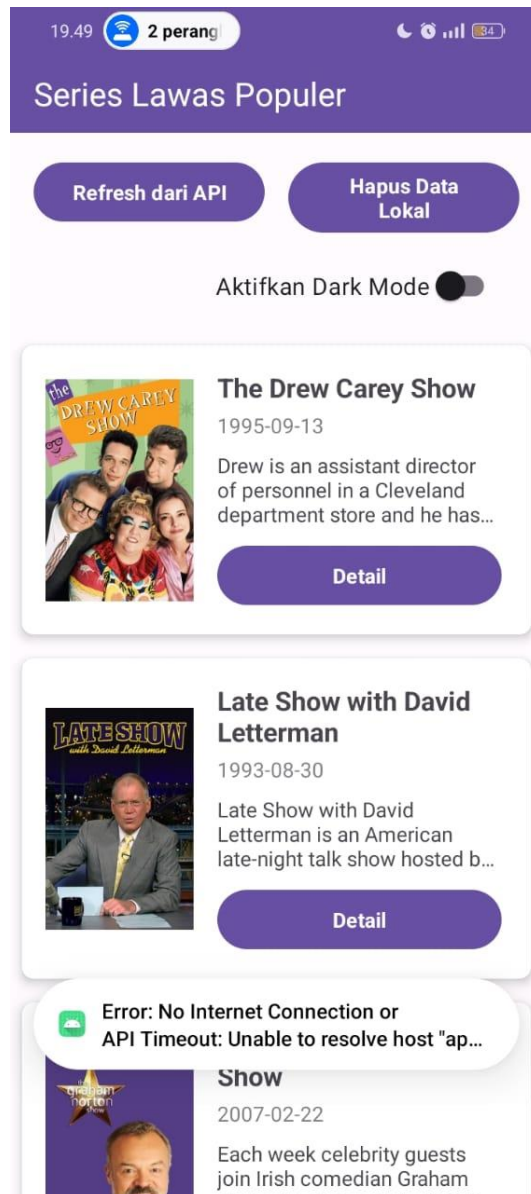
Gambar 2 Halaman Detail



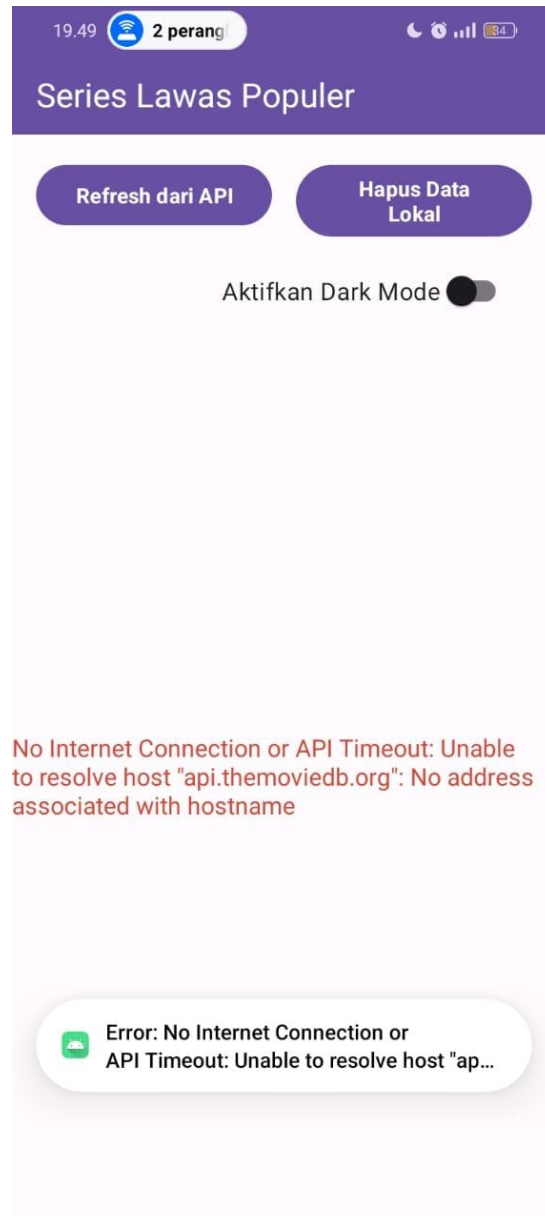
Gambar 3 Halaman Utama DarkMode



Gambar 4 Halaman Detail DarkMode



Gambar 5 Halaman Utama Jika Offline



C. Pembahasan

1. DramaDao.kt

Pada baris 1 package com.example.connectapi.data.local.dao menunjukkan bahwa file Kotlin ini berada di dalam paket dao di bawah domain com.example.connectapi.data.local. Baris 3-8 merupakan impor agar file ini bisa menggunakan kelas-kelas yang diperlukan dari Android Jetpack Room (seperti Dao, Insert, OnConflictStrategy, dan Query) serta Flow dari Kotlin Coroutines. Baris 11 mendefinisikan antarmuka (interface) bernama DramaDao yang akan digunakan sebagai Data Access Object (DAO) untuk berinteraksi dengan database Room. Baris 12-13 adalah fungsi insertDramas yang berfungsi untuk menyisipkan daftar DramaEntity ke dalam database. Anotasi @Insert menandai fungsi ini sebagai operasi penyisipan, dan onConflict = OnConflictStrategy.REPLACE berarti jika ada konflik (misalnya, data dengan primary key yang sama sudah ada), data yang lama akan diganti dengan yang baru. Baris 15-16 adalah fungsi getAllDramas yang berfungsi untuk mendapatkan semua data drama dari tabel dramas. Anotasi @Query("SELECT * FROM dramas") menunjukkan bahwa fungsi ini akan menjalankan kueri SQL untuk memilih semua kolom dari tabel dramas. Fungsi ini mengembalikan Flow<List<DramaEntity>>, yang berarti data akan dikirim secara asinkron dan aplikasi akan menerima pembaruan setiap kali data di tabel berubah. Baris 18-19 adalah fungsi deleteAllDramas yang berfungsi untuk menghapus semua data dari tabel dramas. Anotasi @Query("DELETE FROM dramas") menunjukkan bahwa fungsi ini akan menjalankan kueri SQL untuk menghapus semua baris dari tabel dramas.

2. DramaDatabase.kt

Pada baris 1 package
com.example.connectapi.data.local.Database
menunjukkan bahwa file Kotlin ini berada di dalam paket Database di
bawah domain com.example.connectapi.data.local. Baris 2-7
merupakan impor agar file ini bisa menggunakan kelas-kelas yang
diperlukan dari Android (Context), Android Jetpack Room (Database,
Room, RoomDatabase), serta DAO (DramaDao) dan entitas
(DramaEntity) yang telah didefinisikan sebelumnya. Baris 9
mendefinisikan kelas abstrak bernama DramaDatabase yang merupakan
database Room. Anotasi @Database mengonfigurasi database ini,
menyebutkan DramaEntity sebagai entitas yang digunakan, versi
database adalah 1, dan exportSchema diatur ke false (yang berarti
skema database tidak akan diekspor). Baris 12 adalah fungsi abstrak
dramaDao() yang mengembalikan instance dari DramaDao. Room akan
secara otomatis menyediakan implementasi untuk fungsi ini. Baris 14-27
adalah bagian companion object yang berisi logika untuk membuat
dan mengelola instance tunggal dari DramaDatabase (pola Singleton).
Baris 15-16 mendeklarasikan variabel INSTANCE yang bersifat
@Volatile untuk memastikan visibilitas di seluruh thread. Baris 18-26
adalah fungsi getDatabase yang menerima Context dan
mengembalikan instance dari DramaDatabase. Jika INSTANCE sudah
ada, itu akan langsung dikembalikan; jika tidak, blok synchronized
akan membuat instance baru menggunakan Room.databaseBuilder,
membangun database dengan nama "drama_db", dan menyimpannya ke
INSTANCE sebelum mengembalikannya.

3. DramaEntity.kt

Pada baris 1 package
com.example.connectapi.data.local.entities
menunjukkan bahwa file Kotlin ini berada di dalam paket entities di

bawah domain `com.example.connectapi.data.local`. Baris 3-4 merupakan impor agar file ini bisa menggunakan anotasi `Entity` dan `PrimaryKey` dari Android Jetpack Room. Baris 6 adalah anotasi `@Entity(tableName = "dramas")` yang menandai kelas `DramaEntity` sebagai entitas database Room. Ini berarti setiap instance dari `DramaEntity` akan sesuai dengan sebuah baris dalam tabel bernama "dramas" di database. Baris 7 mendefinisikan data class `DramaEntity`, sebuah kelas data Kotlin yang secara otomatis menyediakan fungsionalitas seperti `equals()`, `hashCode()`, dan `toString()`. Kelas ini merepresentasikan struktur data untuk setiap drama yang akan disimpan dalam database. Baris 8 mendefinisikan properti `id` dengan anotasi `@PrimaryKey`. Ini menandai `id` sebagai kunci utama (primary key) dari tabel "dramas", memastikan setiap entri memiliki identifikasi unik. Baris 9-12 mendefinisikan properti-properti lain dari entitas drama, yaitu `name`, `overview`, `posterPath`, dan `firstAirDate`. Properti `posterPath` dan `firstAirDate` ditandai dengan `?` yang berarti mereka bisa bernilai null.

4. DramaApiService.kt

Pada baris 1 package `com.example.connectapi.data.remote.api` menunjukkan bahwa file Kotlin ini berada di dalam paket `api` di bawah domain `com.example.connectapi.data.remote`. Baris 3-6 merupakan impor agar file ini bisa menggunakan kelas-kelas yang diperlukan dari Retrofit (`Response` dan `http.GET`, `http.Query`) serta model data `PopularDramasResponse` yang telah didefinisikan. Baris 8 mendefinisikan antarmuka (interface) bernama `DramaApiService`. Antarmuka ini digunakan oleh Retrofit untuk mendefinisikan *endpoint* API dan bagaimana aplikasi akan berinteraksi dengannya. Baris 10-14 adalah fungsi `getPopularDramas` yang merupakan metode *suspending function*, artinya bisa dipanggil dari *coroutine*. Fungsi ini ditandai dengan

`@GET("tv/popular")`, yang menunjukkan bahwa ia akan melakukan permintaan HTTP GET ke *endpoint* "tv/popular" dari URL dasar API. Fungsi ini menerima tiga parameter: `apiKey` yang wajib (`@Query("api_key")`), `language` dengan nilai *default* "en-US" (`@Query("language")`), dan `page` dengan nilai *default* 1 (`@Query("page")`). Semua parameter ini akan ditambahkan sebagai *query parameter* ke URL permintaan. Fungsi ini mengembalikan `Response<PopularDramasResponse>`, yang merupakan objek respons dari Retrofit yang berisi data drama populer.

5. DramaDto.kt

Pada baris 1 `package com.example.connectapi.data.remote.model` menunjukkan bahwa file Kotlin ini berada di dalam paket `model` di bawah domain `com.example.connectapi.data.remote`. Baris 3-4 merupakan `import` agar file ini bisa menggunakan anotasi `SerializedName` dan `Serializable` dari library KotlinX Serialization. Baris 6 adalah anotasi `@Serializable` yang menandai kelas `DramaDto` agar bisa di-*serialize* dan di-*deserialize* (diubah dari/ke format data seperti JSON). Baris 7 mendefinisikan data class `DramaDto`, sebuah kelas data Kotlin yang merepresentasikan objek data untuk drama yang diterima dari API. Baris 8-10 mendefinisikan properti `id`, `name`, dan `overview` yang akan langsung dipetakan dari data JSON. Baris 11-14 adalah properti `posterPath` dan `firstAirDate` yang menggunakan anotasi `@SerializedName`. Anotasi ini digunakan untuk memetakan nama properti di JSON ("`poster_path`" dan "`first_air_date`") ke nama properti di kelas Kotlin (`posterPath` dan `firstAirDate`) yang menggunakan *camelCase*. Kedua properti ini juga ditandai dengan `?` yang berarti mereka bisa bernilai `null`.

6. PopularDramasResponse.kt

Pada baris 1 `package com.example.connectapi.data.remote.model` menunjukkan bahwa file Kotlin ini berada di dalam paket `model` di bawah domain `com.example.connectapi.data.remote`. Baris 2-3 merupakan impor agar file ini bisa menggunakan anotasi `SerializedName` dan `Serializable` dari library KotlinX Serialization. Baris 5 adalah anotasi `@Serializable` yang menandai kelas `PopularDramasResponse` agar bisa di-*serialize* dan di-*deserialize* (diubah dari/ke format data seperti JSON). Baris 6 mendefinisikan data class `PopularDramasResponse`, sebuah kelas data Kotlin yang merepresentasikan struktur respons lengkap dari API ketika meminta daftar drama populer. Baris 7-8 mendefinisikan properti `page` yang akan memetakan ke nama `page` dari JSON. Baris 9-10 mendefinisikan properti `results` yang akan memetakan ke nama `results` dari JSON, dan ini adalah daftar objek `DramaDto`. Baris 11-12 mendefinisikan properti `totalPages` yang memetakan ke `total_pages` dari JSON. Terakhir, baris 13-14 mendefinisikan properti `totalResults` yang memetakan ke `total_results` dari JSON. Semua properti ini menggunakan anotasi `@SerializedName` untuk memetakan nama properti di JSON ke nama properti di kelas Kotlin yang menggunakan *camelCase*.

7. ApiClient.kt

Pada baris 1 `package com.example.connectapi.data.remote` menunjukkan bahwa file Kotlin ini berada di dalam paket `remote` di bawah domain `com.example.connectapi.data.remote`. Baris 3-12 merupakan impor agar file ini bisa menggunakan kelas-kelas seperti `Log` dari Android, `BuildConfig` (yang berisi variabel konfigurasi proyek), `DramaApiService` (antarmuka API yang telah didefinisikan), `asConverterFactory` dari Retrofit KotlinX Serialization, `Json` dari KotlinX Serialization, `MediaType` dan `OkHttpClient` dari OkHttp,

HttpLoggingInterceptor dari OkHttp Logging, Retrofit dari Retrofit, dan TimeUnit dari Java. Baris 14 mendefinisikan objek ApiClient, yang berfungsi sebagai Singleton untuk menyediakan instance Retrofit dan DramaApiService yang telah dikonfigurasi. Baris 15-16 mendeklarasikan BASE_URL sebagai alamat dasar API dan TAG untuk keperluan *logging*. Baris 18-23 menginisialisasi objek Json dari KotlinX Serialization dengan konfigurasi khusus seperti ignoreUnknownKeys, prettyPrint, isLenient, dan coerceInputValues untuk menangani parsing JSON secara fleksibel. Baris 25-29 mendefinisikan okHttpClient secara *lazy*. OkHttpClient ini dikonfigurasi dengan *timeout* 30 detik untuk koneksi, baca, dan tulis. Terdapat blok try-catch baris 31-43 untuk menambahkan HttpLoggingInterceptor yang berfungsi untuk *logging* permintaan dan respons HTTP, namun penanganan *error* disertakan jika *interceptor* ini tidak ditemukan atau terjadi kesalahan lain. Baris 45-69 mendefinisikan dramaApiService juga secara *lazy*. Di sini, kunci API (TMDB_API_KEY) diambil dari BuildConfig. Ada pemeriksaan untuk memastikan kunci API tidak kosong atau "null", dan jika ada masalah, IllegalStateException akan dilemparkan. Jika kunci API valid, Retrofit dibangun dengan BASE_URL, okHttpClient yang telah dikonfigurasi, dan json.asConverterFactory untuk menangani konversi JSON. Akhirnya, instance dari DramaApiService dibuat dan dikembalikan. Terdapat juga penanganan *error* yang komprehensif untuk menangani IllegalStateException dan Exception lainnya selama inisialisasi API.

8. DramaRepositoryImpl.kt

Pada baris 1 package com.example.connectapi.data.repository menunjukkan bahwa file Kotlin ini berada di dalam paket repository di bawah domain com.example.connectapi.data.. Baris 3-19 merupakan impor

agar file ini bisa menggunakan kelas-kelas seperti `Log` dari `Android`, `DramaDao` dan `DramaEntity` dari lokal database, `DramaApiService` dari remote API, `Drama` dari domain model, `DramaRepository` dari domain repository, `Resource` dari utilitas, `toDomainModel` dan `toEntity` untuk pemetaan data, serta berbagai fungsi `Flow` dan `Dispatchers` dari `Kotlin Coroutines` dan `Retrofit`.

Baris 21 mendefinisikan kelas `DramaRepositoryImpl` yang mengimplementasikan antarmuka `DramaRepository`. Kelas ini bertanggung jawab untuk mengelola data drama, baik dari sumber lokal (database `Room`) maupun remote (API `The Movie Database`). Konstruktor kelas ini menerima `dramaDao` untuk akses database, `dramaApiService` untuk panggilan API, dan `apiKey` yang diperlukan untuk otentikasi API. Baris 27 adalah implementasi fungsi `getPopularDramas()` yang mengembalikan `Flow` dari `Resource<List<Drama>>`. Fungsi ini dimulai dengan memancarkan `Resource.Loading()`. Kemudian, mencoba mengambil data drama dari database lokal (cache). Jika data cache ditemukan dan tidak kosong, ia akan memancarkan `Resource.Success` dengan data cache tersebut dan mencatatnya. Jika cache kosong, ia akan mencatatnya juga. Selanjutnya, fungsi ini mencoba mengambil data drama terbaru dari API. Kemudian, ia memanggil `getPopularDramas` dari `dramaApiService`. Jika respons berhasil (`response.isSuccessful`), data DTO dari respons dikonversi ke model domain, kemudian database lokal dihapus dan diisi ulang dengan data terbaru dari API menggunakan `withContext(Dispatchers.IO)` untuk operasi I/O asinkron. Setelah itu, `Resource.Success` dipancarkan dengan data terbaru dan dicatat pada baris 52. Jika respons API tidak berhasil baris 58-70 `Resource.Error` akan dipancarkan dengan pesan kesalahan API, dan jika ada, data cache akan disertakan kembali. Terakhir, terdapat blok `catch` yang menangani berbagai jenis kesalahan seperti `HttpException` (kesalahan jaringan HTTP), `IOException` (masalah koneksi internet atau

timeout API), dan *Exception* umum lainnya, di mana *Resource.Error* akan dipancarkan dengan pesan kesalahan yang relevan dan data cache jika tersedia. Baris 72-77 adalah fungsi *insertDramas* yang merupakan *suspending function* untuk menyisipkan daftar *Drama* ke database lokal, mengubahnya menjadi *DramaEntity* dan mencatat keberhasilannya. Ini dilakukan di *Dispatchers.IO*. Baris 78-84 adalah fungsi *clearAllDramas* yang juga merupakan *suspending function* untuk menghapus semua data drama dari database lokal dan mencatatnya. Ini juga dijalankan di *Dispatchers.IO*.

9. DomainMappers.kt

Pada baris 1 `package com.example.connectapi.utils` menunjukkan bahwa file Kotlin ini berada di dalam paket `utils` di bawah domain `com.example.connectapi`. Baris 3-5 merupakan impor agar file ini bisa menggunakan kelas-kelas seperti *DramaEntity* dari entitas lokal, *DramaDto* dari model remote, dan *Drama* dari domain model. Baris 7-15 mendefinisikan fungsi ekstensi `toDomainModel()` untuk objek *DramaDto*. Fungsi ini bertugas mengonversi objek *DramaDto* (yang merepresentasikan data dari API) menjadi objek *Drama* (model domain aplikasi), dengan memetakan properti-propertinya secara langsung. Baris 16-24 mendefinisikan fungsi ekstensi `toDomainModel()` kedua untuk objek *DramaEntity*. Fungsi ini bertugas mengonversi objek *DramaEntity* (yang merepresentasikan data dari database lokal) menjadi objek *Drama* (model domain aplikasi), juga dengan memetakan properti-propertinya secara langsung. Baris 25-33 mendefinisikan fungsi ekstensi `toEntity()` untuk objek *DramaDto*. Fungsi ini bertanggung jawab untuk mengonversi objek *DramaDto* menjadi objek *DramaEntity* agar data dari API dapat disimpan ke dalam database lokal. Baris 34-42 mendefinisikan fungsi ekstensi `toEntity()` terakhir untuk objek *Drama*. Fungsi ini bertugas mengonversi objek *Drama* (model domain aplikasi)

menjadi objek `DramaEntity`, yang diperlukan ketika data dari domain akan disimpan atau diperbarui di database lokal.

10. Drama.kt

Pada baris 1 package `com.example.connectapi.domain.model` menunjukkan bahwa file Kotlin ini berada di dalam paket `model` di bawah domain `com.example.connectapi.domain`. Baris 3-4 merupakan impor agar file ini bisa menggunakan antarmuka `Parcelable` dari Android OS dan anotasi `Parcelize` dari Kotlin `Parcelize`. Baris 6 adalah anotasi `@Parcelize` yang memungkinkan objek dari kelas `Drama` untuk dilewatkan antar komponen Android (seperti `Activity` atau `Fragment`) dengan mudah, tanpa perlu mengimplementasikan metode `Parcelable` secara manual. Baris 7 mendefinisikan data `class Drama`, sebuah kelas data Kotlin yang merepresentasikan model domain untuk sebuah drama. Ini adalah representasi bersih dari data drama yang digunakan di seluruh lapisan domain aplikasi. Baris 8-12 mendefinisikan properti-properti utama dari drama: `id` sebagai identifikasi unik, `name` sebagai judul, `overview` sebagai ringkasan cerita, `posterPath` sebagai jalur gambar poster (yang bisa bernilai `null`), dan `firstAirDate` sebagai tanggal tayang perdana (yang juga bisa bernilai `null`). Terakhir, `:` `Parcelable` pada baris 13 menunjukkan bahwa kelas `Drama` mengimplementasikan antarmuka `Parcelable`, memungkinkan objeknya untuk di-*serialize* dan di-*deserialize* dengan efisien.

11. DramaRepository.kt

Pada baris 1 package `com.example.connectapi.domain.repository` menunjukkan bahwa file Kotlin ini berada di dalam paket `repository` di bawah domain `com.example.connectapi.domain`. Baris 3-5 merupakan impor

agar file ini bisa menggunakan kelas `Drama` dari model domain, kelas `Resource` dari utilitas, dan `Flow` dari Kotlin Coroutines.

Baris 7 mendefinisikan sebuah antarmuka (interface) bernama `DramaRepository`. Antarmuka ini berfungsi sebagai kontrak yang mendefinisikan operasi-operasi yang harus disediakan oleh repositori drama. Baris 8-9 adalah deklarasi fungsi `getPopularDramas()`. Fungsi ini mengembalikan `Flow<Resource<List<Drama>>>`, yang berarti ia akan menyediakan *stream* data drama populer yang dibungkus dalam objek `Resource`. Objek `Resource` ini memungkinkan representasi berbagai status (seperti *loading*, sukses, atau *error*) selama pengambilan data, dan `Flow` memungkinkan pengamatan perubahan data secara asinkron.

12. GetDramaUseCase

Pada baris 1 package `com.example.connectapi.domain.usecase` menunjukkan bahwa file Kotlin ini berada di dalam paket `usecase` di bawah domain `com.example.connectapi.domain`. Baris 3-5 merupakan impor agar file ini bisa menggunakan kelas `Drama` dari model domain, `DramaRepository` dari repository domain, dan `Resource` dan `Flow` dari Kotlin Coroutines. Baris 8 mendefinisikan kelas `GetDramaUseCase`. Kelas ini adalah *use case* atau *interactor* yang berfungsi sebagai jembatan antara lapisan UI/ViewModel dan lapisan repositori. Kelas ini menerima `dramaRepository` sebagai dependensinya melalui konstruktor. Baris 9-10 mendefinisikan fungsi `invoke()` dengan *operator* `fun invoke()`. Ini memungkinkan objek `GetDramaUseCase` untuk dipanggil seperti sebuah fungsi (misalnya, `getDramaUseCase()`). Fungsi ini mengembalikan `Flow<Resource<List<Drama>>>`, yang diperoleh langsung dari `dramaRepository.getPopularDramas()`. Dengan begitu, `GetDramaUseCase` hanya meneruskan permintaan pengambilan drama

populer dari repositori, menjadikannya sebuah *use case* yang bersih dan tunggal dalam fungsinya.

13. DetailFragment.kt

Pada baris 1 package com.example.connectapi.presentation.ui menunjukkan bahwa file Kotlin ini berada di dalam paket ui di bawah domain com.example.connectapi.presentation. Baris 3-10 merupakan impor agar file ini bisa menggunakan kelas-kelas seperti Bundle, LayoutInflater, View, ViewGroup dari Android, Fragment dari AndroidX, load dari Coil, FragmentDetailBinding (binding class yang dihasilkan dari View Binding), dan Drama dari model domain. Baris 12 mendefinisikan kelas DetailFragment yang merupakan turunan dari Fragment, berfungsi untuk menampilkan detail sebuah drama. Baris 13-14 mendeklarasikan properti `_binding` sebagai variabel pembantu untuk menyimpan instance dari FragmentDetailBinding yang bisa bernilai null, serta properti `binding` sebagai *getter* yang tidak-null untuk memudahkan akses ke binding. Baris 16-22 adalah *override* dari fungsi `onCreateView()`. Fungsi ini bertanggung jawab untuk meng-inflasi (membuat) tampilan antarmuka pengguna fragment. Di sini, `FragmentDetailBinding.inflate()` digunakan untuk meng-inflasi layout dan mengikat tampilan ke objek binding, yang kemudian disimpan ke `_binding`, dan akar tampilan dari binding (`binding.root`) dikembalikan. Baris 23-35 adalah *override* dari fungsi `onViewCreated()`. Fungsi ini dipanggil setelah `onCreateView()` selesai dan tampilan fragment telah dibuat dan mencoba mengambil objek Drama yang dikirimkan sebagai argumen ke fragment dengan kunci "drama". Jika objek drama ditemukan dan tidak null properti-properti drama seperti `name`, `overview`, dan `firstAirDate` akan diatur ke `TextView` yang sesuai. `posterPath` digunakan bersama library Coil

(coil.load) untuk memuat gambar poster dari URL dan menampilkannya di ImageView dengan efek *crossfade*. Terakhir, baris 33-36 adalah *override* dari fungsi `onDestroyView()`. Fungsi ini dipanggil ketika tampilan fragment akan dihancurkan. Pada baris 36, `_binding` diatur kembali menjadi `null` untuk membantu menghindari *memory leak* karena referensi ke tampilan tidak lagi dibutuhkan.

14. DramaAdapter.kt

Pada baris 1 package `com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah `com.example.connectapi.presentation`. Baris 3-10 mengimpor kelas-kelas penting seperti `LayoutInflater`, `ViewGroup`, `DiffUtil`, `ListAdapter`, `RecyclerView`, `coil.load` untuk memuat gambar, `ItemDramaBinding` yang dihasilkan dari View Binding, serta `Drama` sebagai model data. Baris 12 mendefinisikan kelas `DramaAdapter` yang berfungsi sebagai jembatan antara data drama dan `RecyclerView` untuk menampilkannya. Adapter ini mewarisi `ListAdapter` yang secara efisien mengelola daftar data dan menggunakan `DramaDiffCallback` untuk mengoptimalkan pembaruan daftar. Baris 14-15 mendeklarasikan dua *callback* opsional, `onDetailClickListener` dan `onNetflixClickListener`, yang memungkinkan aksi tertentu dijalankan ketika tombol detail atau Netflix diklik untuk sebuah drama. Baris 17-20 mengimplementasikan `onCreateViewHolder`, yang bertanggung jawab untuk membuat dan mengembalikan sebuah `DramaViewHolder` baru dengan meng-inflasi tata letak `ItemDramaBinding` untuk setiap item daftar. Baris 22-25 mengimplementasikan `onBindViewHolder`, yang bertugas mengikat data `Drama` tertentu ke tampilan di dalam `DramaViewHolder` pada posisi yang diberikan. Baris 27-42 mendefinisikan kelas internal `DramaViewHolder`. Kelas ini memegang referensi ke tampilan setiap

item di RecyclerView. Fungsi bind mengatur teks untuk tvTitle, tvOverview, dan tvReleaseDate dari objek Drama. Selain itu, ia menggunakan Coil untuk memuat gambar poster ke ivPoster dan mengatur OnClickListener pada btnDetail untuk memicu onDetailClickListener ketika tombol diklik. Terakhir, baris 43 mendefinisikan DramaDiffCallback, sebuah kelas utilitas yang membantu ListAdapter menentukan perbedaan antara dua daftar drama. Baris 44-46 fungsi areItemsTheSame, membandingkan id dari dua objek Drama untuk mengetahui apakah mereka merujuk pada item yang sama. Baris 48-52, fungsi areContentsTheSame, membandingkan seluruh isi objek Drama untuk mengetahui apakah ada perubahan pada data item tersebut.

15. DramaListFragment

Pada baris 1 package com.example.connectapi.presentation.ui menunjukkan bahwa file Kotlin ini berada di dalam paket ui di bawah com.example.connectapi.presentation. Baris 3-26 mengimpor berbagai kelas dan fungsi penting, termasuk komponen Android UI (Bundle, Log, LayoutInflater, View, ViewGroup, Toast), komponen Fragment (Fragment, viewModels), manajemen siklus hidup (Lifecycle, lifecycleScope, repeatOnLifecycle), navigasi (findNavController), RecyclerView (LinearLayoutManager), konfigurasi aplikasi (BuildConfig), sumber daya (R), komponen database (DramaDatabase), klien API (ApiClient), implementasi repositori (DramaRepositoryImpl), use case (GetDramaUseCase), kelas utilitas (Resource, AppPreferences), dan tema (AppCompatDelegate), serta launch untuk coroutines. Baris 28 mendefinisikan kelas DramaListFragment sebagai turunan dari Fragment, yang bertanggung jawab untuk menampilkan daftar drama. Baris 29-30 mendeklarasikan variabel

`_binding` untuk View Binding dan `binding` sebagai *getter* yang aman. Baris 32-43 menginisialisasi `dramaViewModel` menggunakan *delegate* `viewModels` dan `DramaViewModelFactory` khusus, di mana dependensi seperti `DramaDao`, `DramaApiService`, `apiKey`, `DramaRepositoryImpl`, dan `GetDramaUseCase` disediakan. Baris 38-39 mendeklarasikan `dramaAdapter` untuk `RecyclerView` dan `appPreferences` untuk pengelolaan preferensi aplikasi. Baris 45-51 mengimplementasikan `onCreateView`, di mana View Binding (`FragmentDramaListBinding.inflate`) digunakan untuk meng-inflasi tata letak fragment dan mengembalikan tampilan utamanya. Baris 52-75 mengimplementasikan `onViewCreated`, yang dipanggil setelah tampilan fragment dibuat. Di sini, `appPreferences` diinisialisasi, `setupRecyclerView()` dan `observeViewModel()` dipanggil untuk menyiapkan daftar dan mengamati data. Selain itu, status *dark mode* dimuat dari preferensi dan diterapkan ke `darkModeSwitch`, dengan *listener* yang menyimpan status *dark mode* yang diubah dan menerapkan tema. Terdapat juga *listener* untuk tombol *refresh* dan *clear local data*. Baris 76-91 mendefinisikan `setupRecyclerView()` yang menginisialisasi `dramaAdapter`, mengatur `LinearLayoutManager` untuk `rvDrama`, dan menetapkan adapter tersebut. Fungsi ini juga mengatur `onDetailClickListener` untuk menavigasi ke `DetailFragment` dengan data drama yang dipilih, serta `onNetflixClickListener` yang menampilkan `Toast`. Baris 92-136 mendefinisikan `observeViewModel()`, yang menggunakan `viewLifecycleOwner.lifecycleScope.launch` dan `repeatOnLifecycle(Lifecycle.State.STARTED)` untuk mengumpulkan data dari `dramaViewModel.popularDramasState`. Blok `when` kemudian menangani tiga status `Resource`: `Loading` (menampilkan `progressBar` atau data `cache`), `Success` (menyembunyikan *loading* dan menampilkan data), dan `Error`

(menyembunyikan *loading*, menampilkan pesan error, Toast, dan jika ada, menampilkan data cache).

Baris 137-147 mendefinisikan `clearLocalData()`, sebuah fungsi suspend yang dipanggil ketika tombol clear data lokal ditekan. Fungsi ini mencoba memanggil `dramaViewModel.clearLocalData()` untuk menghapus data, menampilkan Toast keberhasilan atau kegagalan, dan mencatat *error*. Baris 148-155 mendefinisikan `applyTheme()`, yang mengubah tema aplikasi menjadi *dark mode* (`AppCompatDelegate.MODE_NIGHT_YES`) atau *light mode* (`AppCompatDelegate.MODE_NIGHT_NO`) berdasarkan nilai `isDarkMode`, dan kemudian memanggil `requireActivity().recreate()` untuk menerapkan perubahan tema secara instan. Terakhir, baris 156-160 mengimplementasikan `onDestroyView()` untuk membersihkan referensi `_binding` menjadi null guna menghindari *memory leak* ketika tampilan fragment dihancurkan.

16. DramaViewModel

Pada baris 1 package `com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah `com.example.connectapi.presentation`. Baris 3-12 mengimpor kelas-kelas penting seperti `ViewModel` dan `viewModelScope` dari `AndroidX Lifecycle`, `Drama` dari `model domain`, `DramaRepository` dari `repository domain`, `GetDramaUseCase` dari `use case domain`, `Resource` dari `utilitas`, serta `MutableStateFlow`, `StateFlow`, `asStateFlow`, dan `launch` dari `Kotlin Coroutines`. Baris 14 mendefinisikan kelas `DramaViewModel` yang mewarisi `ViewModel`, bertugas mengelola dan menyediakan data drama ke UI dengan siklus hidup yang sadar akan Android. Konstruktornya menerima `getDramaUseCase` dan `dramaRepository` sebagai dependensi. Baris 19-20

mendeklarasikan `_popularDramasState` sebagai `MutableStateFlow` yang menyimpan status data drama populer, diinisialisasi dengan `Resource.Loading()`. Kemudian, `popularDramasState` dideklarasikan sebagai `StateFlow` publik yang hanya-baca, diekspos sebagai `_popularDramasState.asStateFlow()`. Baris 22-24 adalah blok `init` yang akan dijalankan saat `DramaViewModel` pertama kali dibuat, memanggil `fetchPopularDramas()` untuk segera memuat data drama. Fungsi `fetchPopularDramas()` menggunakan `viewModelScope.launch` untuk meluncurkan *coroutine* dan mengumpulkan data dari `getDramaUseCase()`, yang kemudian membarui nilai `_popularDramasState` setiap kali ada perubahan. Baris 32-34 mendefinisikan fungsi `refreshDramasFromLocal()`, yang cukup memanggil kembali `fetchPopularDramas()` untuk memicu pengambilan ulang data (termasuk dari lokal) dan memperbarui UI. Baris 36-42 mendefinisikan fungsi `clearLocalData()`. Fungsi ini juga meluncurkan *coroutine* menggunakan `viewModelScope.launch`. Di dalamnya, ia mencoba meng-*casting* `dramaRepository` ke `DramaRepositoryImpl` dan jika berhasil, memanggil `clearAllDramas()` untuk menghapus data lokal. Setelah itu, `fetchPopularDramas()` dipanggil kembali untuk memperbarui tampilan dengan data yang mungkin kosong atau data baru dari API jika ada.

17. DramaViewModelFactory.kt

Pada baris 1 package `com.example.connectapi.presentation.ui` menunjukkan bahwa file Kotlin ini berada di dalam paket `ui` di bawah `com.example.connectapi.presentation`. Baris 3-10 mengimpor kelas-kelas seperti `Application` dari `Android`, `ViewModel` dan `ViewModelProvider` dari `AndroidX Lifecycle`, `DramaDatabase`

dari lokal database, ApiClient dari remote API, DramaRepositoryImpl dari implementasi repositori, GetDramaUseCase dari *use case* domain, dan BuildConfig dari konfigurasi proyek. Baris 12 mendefinisikan kelas DramaViewModelFactory yang mengimplementasikan ViewModelProvider.Factory. Kelas ini berfungsi sebagai *pabrik* untuk membuat instance DramaViewModel dengan dependensi yang diperlukan. Konstruktornya menerima application dan getPopularDramaUseCase sebagai dependensi awal. Baris 15 adalah implementasi fungsi create(), yang wajib di-override dari ViewModelProvider.Factory. Fungsi ini bertanggung jawab untuk membuat dan mengembalikan instance ViewModel yang diminta. Pada baris 17, ia memeriksa apakah modelClass yang diminta adalah DramaViewModel. Jika sesuai, pada baris 19-23, secara manual menginisialisasi semua dependensi yang dibutuhkan oleh DramaViewModel: dramaDao dari DramaDatabase, dramaApiService dari ApiClient, apiKey dari BuildConfig, dramaRepository dengan dependensi yang telah dibuat, dan getDramaUseCase dengan dramaRepository. Setelah semua dependensi siap, pada baris 25, membuat dan mengembalikan instance DramaViewModel dengan getDramaUseCase dan dramaRepository yang telah disiapkan, dengan *annotation* @SuppressWarnings("UNCHECKED_CAST") untuk menghindari peringatan *casting*. Jika modelClass yang diminta bukan DramaViewModel, pada baris 28, ia akan melempar IllegalArgumentException.

18. MainActivity.kt

Pada baris 1 package com.example.connectapi.presentation.ui menunjukkan bahwa file Kotlin ini berada di dalam paket ui di bawah com.example.connectapi.presentation. Baris 3-9 mengimpor

kelas-kelas penting seperti `Bundle` dari `Android OS`, `AppCompatActivity` dan `AppCompatActivity` dari `AndroidX`, `AppCompatActivity`, `NavHostFragment` dan `setupActionBarWithNavController` dari `AndroidX Navigation`, `ActivityMainBinding` yang dihasilkan dari `View Binding`, dan `AppPreferences` dari utilitas. Baris 11 mendefinisikan kelas `MainActivity` yang merupakan *activity* utama aplikasi ini dan mewarisi `AppCompatActivity`. Baris 12-13 mendeklarasikan binding untuk `View Binding` dan `appPreferences` untuk mengelola preferensi pengguna. Baris 15-29 mengimplementasikan fungsi `onCreate()`, yang dijalankan saat *activity* pertama kali dibuat. Pada baris 17-18, `ActivityMainBinding.inflate()` digunakan untuk meng-inflasi *layout* `activity_main` dan mengaturnya sebagai tampilan konten menggunakan `setContentView()`. Baris 20 menginisialisasi `appPreferences`. Baris 22 memanggil `applySavedTheme()` untuk menerapkan tema yang terakhir disimpan. Baris 24 menyiapkan bilah aksi (*toolbar*) dengan mengaturnya sebagai *support action bar*, menemukan `NavHostFragment`, dan mengikat `NavController` ke bilah aksi untuk integrasi navigasi. Baris 31-36 mengimplementasikan `onSupportNavigateUp()`, sebuah fungsi yang dipanggil ketika tombol 'Up' di bilah aksi ditekan. Fungsi ini menangani navigasi kembali dalam tumpukan navigasi menggunakan `NavController.navigateUp()`. Baris 37-44 mendefinisikan fungsi `applySavedTheme()`. Fungsi ini mengambil status *dark mode* yang disimpan dari `appPreferences` dan kemudian menggunakan `AppCompatActivity.setDefaultNightMode()` untuk mengatur tema aplikasi ke mode terang atau gelap sesuai dengan status yang disimpan.

19. AppPreferences

Pada baris 1 `package com.example.connectapi.utils` menunjukkan bahwa file Kotlin ini berada di dalam paket `utils` di bawah

`com.example.connectapi`. Baris 3-4 mengimpor kelas-kelas penting seperti `Context` dan `SharedPreferences` dari `Android`.

Baris 6 mendefinisikan kelas `AppPreferences`, yang dirancang untuk mengelola preferensi aplikasi, khususnya untuk menyimpan status *dark mode*. Konstruktor kelas ini menerima objek `Context` yang diperlukan untuk mengakses `SharedPreferences`. Baris 7-9 mendeklarasikan konstanta `PREFS_NAME` sebagai nama file preferensi (`"app_prefs"`), `KEY_DARK_MODE` sebagai kunci untuk menyimpan status *dark mode* (`"dark_mode"`), serta properti `sharedPrefs` untuk menyimpan instance dari `SharedPreferences`. Baris 11-13 adalah blok `init`, yang dijalankan saat objek `AppPreferences` dibuat. Di sini, `sharedPrefs` diinisialisasi dengan memanggil `context.getSharedPreferences()`, menggunakan `PREFS_NAME` dan `Context.MODE_PRIVATE` untuk memastikan preferensi ini hanya dapat diakses oleh aplikasi ini. Baris 14-16 mendefinisikan fungsi `saveDarkModeState()`. Fungsi ini menerima nilai *boolean* `isDarkMode` dan menggunakannya untuk menyimpan status *dark mode* ke `SharedPreferences` di bawah `KEY_DARK_MODE`. Metode `.apply()` digunakan untuk menyimpan perubahan secara asinkron. Terakhir, baris 17-19 mendefinisikan fungsi `getDarkModeState()`. Fungsi ini mengambil nilai *boolean* yang terkait dengan `KEY_DARK_MODE` dari `SharedPreferences`. Jika kunci tersebut belum ada, ia akan mengembalikan nilai *default* `false`.

20. Resource.kt

Pada baris 1 `package com.example.connectapi.utils` menunjukkan bahwa file Kotlin ini berada di dalam paket `utils` di bawah `com.example.connectapi`. Baris 3 mendefinisikan kelas `Resource` sebagai *sealed class*. *Sealed class* adalah jenis kelas abstrak yang memungkinkan mendefinisikan himpunan subclass terbatas dalam file yang sama, memastikan semua kemungkinan kasus ditangani. Kelas ini memiliki

dua properti opsional: `data` (generik `T`, yang bisa berupa tipe data apa pun) dan `message` (`String`), yang akan digunakan oleh subclass-nya.

Baris 4 mendefinisikan `Loading` sebagai subclass dari `Resource`. Kelas ini merepresentasikan status di mana data sedang dimuat. Ia memiliki konstruktor yang menerima data opsional, yang bisa digunakan untuk menampilkan data lama saat data baru sedang dimuat. Baris 5 mendefinisikan `Success` sebagai subclass dari `Resource`. Kelas ini merepresentasikan status di mana data telah berhasil diambil. Ia memiliki konstruktor yang wajib menerima data (`T`) yang berhasil dimuat. Baris 6 mendefinisikan `Error` sebagai subclass dari `Resource`. Kelas ini merepresentasikan status di mana terjadi kesalahan saat mengambil data. Ia memiliki konstruktor yang wajib menerima `message` (`String`) yang menjelaskan kesalahan, dan juga bisa menerima data opsional untuk menampilkan data lama jika ada kesalahan.

21. Activity_main.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` adalah deklarasi XML standar. Baris 2 `ConstraintLayout` adalah elemen tata letak akar, yang memungkinkan menempatkan dan mengukur tampilan berdasarkan hubungan relatif antar tampilan. Baris 2-5 mendefinisikan *namespace* XML (`xmlns:android`, `xmlns:app`, `xmlns:tools`) yang digunakan untuk atribut-atribut Android, atribut kustom dari pustaka (`app`), dan atribut khusus *tools* yang hanya untuk pratinjau desain. Atribut `android:layout_width` dan `android:layout_height` disetel ke `match_parent` agar tata letak memenuhi lebar dan tinggi layar penuh, sementara `tools:context` pada baris 6 menunjukkan kelas `MainActivity` sebagai konteks desain untuk pratinjau. Pada baris 9, `<com.google.android.material.appbar.AppBarLayout>` adalah kontainer yang dirancang untuk bilah aplikasi (toolbar) Material Design, memungkinkan efek gulir seperti menyembunyikan atau

menampilkan toolbar saat pengguna menggulir. Atribut `android:layout_width` diatur ke `match_parent` agar memenuhi lebar, `android:layout_height` ke `wrap_content` agar tingginya menyesuaikan kontennya, dan `app:layout_constraintTop_toTopOf="parent"` pada baris 12 menempatkannya di bagian atas layar.

Di dalam `AppBarLayout`, pada baris 14, `<androidx.appcompat.widget.Toolbar>` adalah bilah alat aplikasi yang berfungsi sebagai *action bar*. Atribut `android:id` memberinya pengenal toolbar, `android:layout_width` diatur ke `match_parent` dan `android:layout_height` ke `53dp` untuk dimensinya. `android:background="?attr/colorPrimary"` mengaturnya agar menggunakan warna primer dari tema aplikasi, dan `app:titleTextColor="@android:color/white"` mengatur warna teks judul menjadi putih.

Selanjutnya, pada baris 20, `<androidx.fragment.app.FragmentContainerView>` adalah kontainer yang digunakan untuk menghosting `Fragment` navigasi. Atribut `android:id` memberinya pengenal `nav_host_fragment`, dan `android:name="androidx.navigation.fragment.NavHostFragment"` menentukan bahwa ini adalah *host* untuk grafik navigasi. Atribut `android:layout_width` dan `android:layout_height` disetel ke `0dp` dengan batasan (`app:layout_constraintBottom_toBottomOf`, `app:layout_constraintEnd_toEndOf`, `app:layout_constraintStart_toStartOf`) untuk mengisi sisa ruang setelah `app_bar_layout`. Atribut `app:defaultNavHost="true"` pada baris 24 menandai ini sebagai `NavHost` default yang akan mencegat tombol kembali sistem, dan `app:navGraph="@navigation/nav_graph"` pada baris 27

menghubungkannya ke grafik navigasi yang didefinisikan dalam `res/navigation/nav_graph.xml`.

22. `fragment_detail.xml`

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` merupakan deklarasi XML standar. Baris 2 `<androidx.constraintlayout.widget.ConstraintLayout>` adalah elemen tata letak utama yang berfungsi sebagai wadah untuk tampilan-tampilan lain dalam fragmen, memungkinkannya mengatur posisi elemen berdasarkan hubungan antar elemen. Baris 2-5 mendefinisikan *namespace* XML yang diperlukan untuk atribut-atribut Android, atribut kustom (`app`), dan atribut khusus alat (`tools`) untuk pratinjau desain. Atribut `android:layout_width` dan `android:layout_height` pada baris 6 disetel ke `match_parent` agar tata letak memenuhi seluruh lebar dan tinggi yang tersedia, sedangkan `tools:context` pada baris 7 menunjukkan bahwa *layout* ini terkait dengan kelas `DetailFragment` untuk keperluan *preview* di *editor*.

Pada baris 9, `<ImageView>` dengan `android:id="@+id/iv_detail_poster"` didefinisikan untuk menampilkan gambar poster drama. Gambar ini akan memenuhi lebar (`match_parent`) dan memiliki tinggi tetap `250dp`, dengan `scaleType="centerCrop"` untuk memastikan gambar mengisi area tampilan sambil mempertahankan rasio aspeknya. `android:src` pada baris 12 menyediakan gambar *placeholder* awal, dan batasan `app:layout_constraintTop_toTopOf="parent"`, `app:layout_constraintStart_toStartOf="parent"`, serta `app:layout_constraintEnd_toEndOf="parent"` pada baris 13-15 menempatkan gambar ini di bagian paling atas tata letak dan membentang dari kiri ke kanan.

Selanjutnya, pada baris 17, `<TextView>` dengan `android:id="@+id/tv_detail_title"` digunakan untuk

menampilkan judul drama. Lebar nya diatur ke 0dp dengan batasan horizontal untuk mengisi ruang yang tersedia, tingginya wrap_content, dan memiliki margin 16dp di bagian atas, kiri, dan kanan. Teks *placeholder* "Judul Drama" disetel dengan ukuran 24sp dan gaya bold. Batasan pada baris 24-26 menempatkan TextView ini tepat di bawah iv_detail_poster dan membentangnya secara horizontal.

Pada baris 29, <TextView> berikutnya dengan android:id="@+id/tv_detail_release_date" berfungsi untuk menampilkan tanggal rilis drama. Sama seperti judul, lebar nya 0dp, tingginya wrap_content, dengan margin 8dp di bagian atas dan 16dp di kiri/kanan. Teks *placeholder* "Tanggal Rilis:YYYY-MM-DD" disetel dengan ukuran 16sp dan alpha="0.7" untuk memberikan efek sedikit transparan. Batasan pada baris 37-39 menempatkannya di bawah tv_detail_title dan membentangnya secara horizontal.

Terakhir, dari baris 41, <TextView> dengan android:id="@+id/tv_detail_overview" dirancang untuk menampilkan sinopsis atau deskripsi drama. Lebar dan tingginya disetel ke 0dp dengan batasan, yang berarti ia akan mengisi sisa ruang yang tersedia di tata letak setelah elemen-elemen di atasnya. Memiliki margin 16dp di semua sisi, teks *placeholder* "Deskripsi panjang drama...", ukuran 14sp, dan android:scrollbars="vertical" untuk memungkinkan teks digulir jika terlalu panjang. Batasan pada baris 49-52 menempatkannya di bawah tv_detail_release_date dan membentangnya hingga ke bagian bawah tata letak serta dari kiri ke kanan.

23. fragment_drama_list.xml

Pada baris 1
 <androidx.constraintlayout.widget.ConstraintLayout
 > adalah elemen tata letak utama yang membungkus seluruh tampilan fragment, memungkinkan penempatan elemen berdasarkan hubungan relatif. Atribut android:layout_width dan

`android:layout_height` pada baris 4-5 diatur ke `match_parent` agar tata letak memenuhi seluruh ruang yang tersedia, sedangkan `tools:context` pada baris 6 menunjukkan `DramaListFragment` sebagai konteks untuk pratinjau desain.

Pada baris 9, sebuah `Button` dengan `android:id="@+id/btn_refresh_api"` didefinisikan untuk memicu *refresh* data dari API. Lebar nya disetel ke `0dp` untuk mengisi ruang yang dibatasi oleh `guideline_center`, tingginya `wrap_content`, dan memiliki teks "Refresh dari API". Batasan pada baris 13-17 menempatkannya di bagian atas layar dan di sisi kiri `guideline_center` dengan margin tertentu. Di sampingnya, pada baris 19, `Button` kedua dengan `android:id="@+id/btn_clear_local_data"` berfungsi untuk menghapus data lokal. Lebar nya juga `0dp`, tingginya `wrap_content`, dan teksnya "Hapus Data Lokal". Batasan pada baris 23-27 menempatkannya di samping kanan `guideline_center` sejajar dengan tombol *refresh*.

Pada baris 29, `<androidx.constraintlayout.widget.Guideline>` dengan `android:id="@+id/guideline_center"` adalah elemen pembantu tak terlihat yang membagi tata letak secara vertikal. Atribut `android:orientation="vertical"` dan `app:layout_constraintGuide_percent="0.5"` pada baris 32-33 menempatkannya di tengah lebar layar, membagi ruang untuk kedua tombol secara merata.

Selanjutnya, pada baris 35, `<com.google.android.material.switchmaterial.SwitchMaterial>` dengan `android:id="@+id/darkModeSwitch"` adalah sakelar untuk mengaktifkan atau menonaktifkan mode gelap. Lebar dan tingginya `wrap_content`, memiliki margin atas `8dp`, dan *padding* `8dp`. Teks "Aktifkan Dark Mode" ditampilkan dengan warna yang sesuai

tema (`?attr/colorOnBackground`) dan ukuran 16sp. Batasan pada baris 43-45 menempatkannya di bawah `btn_clear_local_data` dan sedikit ke kanan dari tengah layar.

Pada baris 48, `<ProgressBar>` dengan `android:id="@+id/progressBar"` adalah indikator *loading*. Lebar dan tingginya `wrap_content`, dan awalnya `visibility="gone"` (tersembunyi). `indeterminateTint="?attr/colorPrimary"` pada baris 51 mengatur warna indikator sesuai tema. Batasan pada baris 52-55 menempatkannya di tengah layar, di bawah `darkModeSwitch`.

Pada baris 57, `<androidx.recyclerview.widget.RecyclerView>` dengan `android:id="@+id/rvDrama"` berfungsi untuk menampilkan daftar drama. Lebar dan tingginya disetel ke `0dp` dengan batasan, memungkinkannya mengisi seluruh sisa ruang di bawah `darkModeSwitch` hingga ke bagian bawah layar. `paddingTop="8dp"` memberikan sedikit ruang di bagian atas daftar, dan `tools:listitem` pada baris 65 menunjukkan *layout* item (`@layout/item_drama`) yang akan digunakan untuk pratinjau.

Terakhir, pada baris 67, `<TextView>` dengan `android:id="@+id/tvError"` digunakan untuk menampilkan pesan kesalahan. Lebar dan tingginya `wrap_content`, teks *placeholder* "Terjadi kesalahan!", warna teks `textColor="@color/red_error"`, dan ukuran 16sp. Awalnya juga `visibility="gone"` (tersembunyi). Batasan pada baris 73-76 menempatkannya di tengah layar, di bawah `darkModeSwitch`, di posisi yang sama dengan `progressBar` dan `rvDrama`, dan akan ditampilkan ketika ada *error*.

24. drama_item_1.xml

Pada baris 1 `<androidx.cardview.widget.CardView>` adalah elemen utama yang berfungsi sebagai wadah untuk setiap item drama dalam daftar, memberikan tampilan kartu dengan sudut membulat dan elevasi

bayangan. Atribut `android:layout_width` disetel ke `match_parent` agar kartu memenuhi lebar yang tersedia, `android:layout_height` ke `wrap_content` agar tingginya menyesuaikan konten, dan `android:layout_margin="8dp"` pada baris 7 memberikan jarak antar kartu. `app:cardCornerRadius="8dp"` dan `app:cardElevation="4dp"` pada baris 8-9 mengatur radius sudut dan elevasi bayangan kartu. Di dalam `CardView`, pada baris 11, `<androidx.constraintlayout.widget.ConstraintLayout>` berfungsi sebagai tata letak internal kartu, memungkinkan penempatan elemen secara fleksibel. Ia mengisi lebar dan tinggi `CardView` dan memiliki *padding* 16dp di semua sisi. Pada baris 16, sebuah `ImageView` dengan `android:id="@+id/iv_poster"` didefinisikan untuk menampilkan poster drama. Lebar dan tingginya disetel ke 100dp dan 150dp secara spesifik, dengan `scaleType="centerCrop"` untuk mengisi area tanpa distorsi. `android:src` pada baris 19 menyediakan gambar *placeholder*, dan batasan `app:layout_constraintStart_toStartOf="parent"`, `app:layout_constraintTop_toTopOf="parent"`, serta `app:layout_constraintBottom_toBottomOf="parent"` pada baris 20-22 menempatkan gambar di sisi kiri kartu dan memusatkannya secara vertikal.

Selanjutnya, pada baris 24, `<TextView>` dengan `android:id="@+id/tv_title"` digunakan untuk menampilkan judul drama. Lebarnya disetel ke 0dp dengan batasan horizontal, tingginya `wrap_content`, dan memiliki margin 16dp dari poster serta 8dp dari akhir. Teks *placeholder* "Judul Drama" disetel dengan ukuran 18sp dan gaya bold. Batasan pada baris 31-33 menempatkan `TextView` ini di samping kanan `iv_poster` dan di bagian atas `ConstraintLayout`. Pada baris 35, `<TextView>` berikutnya dengan `android:id="@+id/tv_release_date"` menampilkan tanggal

rilis. Lebar nya 0dp, tingginya wrap_content, dan memiliki margin yang spesifik. Teks *placeholder* "Tanggal Rilis:YYYY-MM-DD" disetel dengan ukuran 14sp dan alpha="0.7" untuk efek sedikit transparan. Batasan pada baris 42-44 menempatkannya di bawah tv_title dan di samping kanan iv_poster. Dari baris 46, <TextView> dengan android:id="@+id/tv_overview" digunakan untuk sinopsis drama. Lebar nya 0dp, tingginya wrap_content, dan memiliki margin. Atribut maxLines="3" dan ellipsize="end" membatasi teks hingga tiga baris, menambahkan elipsis jika teks terlalu panjang. Teks *placeholder* "Ini adalah ringkasan singkat dari drama..." disetel dengan ukuran 14sp. Batasan pada baris 53-55 menempatkannya di bawah tv_release_date dan di samping kanan iv_poster. Terakhir, pada baris 57, <LinearLayout> berfungsi sebagai wadah horizontal untuk tombol-tombol. Lebar nya 0dp dengan batasan, tingginya wrap_content, dan memiliki margin atas 8dp serta margin kiri 16dp. Batasan pada baris 60-63 menempatkannya di bawah tv_overview dan di samping iv_poster, membentang hingga akhir kartu. Di dalamnya, pada baris 65, sebuah Button dengan android:id="@+id/btn_detail" didefinisikan dengan lebar 0dp dan layout_weight="1" agar mengisi ruang yang tersedia, serta teks "Detail" dan margin akhir 4dp.

25. Main_menu.xml

Pada baris 1 <?xml version="1.0" encoding="utf-8"?> adalah deklarasi XML standar. Baris 2 <menu> adalah elemen akar yang mendefinisikan struktur menu dalam aplikasi Android. Atribut xmlns:android dan xmlns:app pada baris 2-3 mendefinisikan namespace XML yang diperlukan untuk atribut-atribut Android standar dan atribut khusus dari pustaka dukungan Android (seperti AppCompatActivity). Pada baris 4 sebuah <item> didefinisikan sebagai entri tunggal dalam menu. Atribut android:id="@+id/action_dark_mode_switch" pada

baris 5 memberikan identifikasi unik untuk item menu ini. Atribut `android:title="Dark Mode"` pada baris 6 mengatur teks yang akan ditampilkan untuk item menu. `app:actionViewClass="com.google.android.material.switchmaterial.SwitchMaterial"` pada baris 7 menentukan bahwa item menu ini tidak hanya teks, tetapi juga akan menampilkan widget `SwitchMaterial` sebagai tampilan aksinya. Terakhir, `app:showAsAction="always"` pada baris 8 menyarankan agar item menu ini selalu ditampilkan di bilah aplikasi (toolbar) jika ada ruang yang cukup, bukan disembunyikan di menu *overflow*.

26. nav_graph.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` adalah deklarasi XML standar. Baris 2 `<navigation>` merupakan elemen akar yang mendefinisikan grafik navigasi aplikasi, yang bertindak sebagai peta untuk menunjukkan bagaimana pengguna dapat bergerak antar layar (fragment). Atribut `android:id="@+id/nav_graph"` pada baris 5 memberikan pengenalan unik untuk grafik ini, sementara `app:startDestination="@id/dramaListFragment"` pada baris 6 menetapkan `dramaListFragment` sebagai tujuan awal saat aplikasi diluncurkan. Pada baris 8, `<fragment>` pertama dengan `android:id="@+id/dramaListFragment"` mewakili layar daftar drama. Atribut `android:name="com.example.connectapi.presentation.ui.DramaListFragment"` pada baris 10 menghubungkan tujuan navigasi ini dengan kelas Kotlin `DramaListFragment` yang sebenarnya. `android:label="Series Lawas Populer"` pada baris 11 mengatur judul yang mungkin ditampilkan di bilah aksi aplikasi, dan `tools:layout="@layout/fragment_drama_list"` pada baris 12 adalah atribut yang berguna untuk pratinjau desain di Android Studio. Di dalam fragmen ini, `<action>` pada baris 13 mendefinisikan jalur navigasi yang mungkin.

`android:id="@+id/action_dramaListFragment_to_detailFragment"` pada baris 14 memberi pengenalan unik pada aksi ini, dan `app:destination="@id/detailFragment"` pada baris 15 menunjukkan bahwa aksi ini mengarahkan ke `detailFragment`. Selanjutnya, pada baris 17, `<fragment>` kedua dengan `android:id="@+id/detailFragment"` mewakili layar detail drama. Mirip dengan fragmen daftar, `android:name="com.example.connectapi.presentation.ui.DetailFragment"` pada baris 19 menautkannya ke kelas Kotlin `DetailFragment`, `android:label="Drama Detail"` pada baris 20 mengatur judulnya, dan `tools:layout="@layout/fragment_detail"` pada baris 21 membantu dengan pratinjau tata letak. Bagian penting dari fragmen detail ini adalah `<argument>` pada baris 22, di mana `android:name="drama"` pada baris 23 menentukan nama data yang akan diteruskan ke fragmen ini, dan `app:argType="com.example.connectapi.domain.model.Drama"` pada baris 24 mendeklarasikan bahwa argumen ini akan menjadi objek dari tipe `Drama`, memungkinkan penerusan detail drama yang dipilih secara aman dari layar daftar ke layar detail.

27. colors.xml

Pada baris 1 `<?xml version="1.0" encoding="utf-8"?>` adalah deklarasi XML standar. Baris 2 `<resources>` merupakan tag pembuka untuk mendefinisikan berbagai sumber daya, termasuk warna, untuk aplikasi Android. Pada baris 4-6, Anda akan menemukan definisi warna dasar seperti `black` (`#FF000000`), `white` (`#FFFFFFFF`), dan `red_error` (`#FFDA4433`), yang sering digunakan sebagai warna fundamental dalam antarmuka pengguna.

Dimulai dari baris 8 hingga 36, terdapat serangkaian definisi warna untuk tema terang (`md_theme_light`) yang mengikuti pedoman Material Design 3 (MD3). Warna-warna ini dikelompokkan berdasarkan peran dalam UI,

seperti `primary` dan `onPrimary` untuk warna merek utama dan teks/ikon di atasnya, `secondary` dan `onSecondary` untuk warna aksen sekunder, `tertiary` dan `onTertiary` untuk warna aksen ketiga, serta warna untuk status seperti `error`, `background`, dan `surface`. Ini memastikan konsistensi visual di seluruh aplikasi pada mode terang.

Selanjutnya, dari baris 38 hingga 66, definisi warna untuk tema gelap (`md_theme_dark_`) disediakan. Warna-warna ini adalah padanan yang dioptimalkan untuk lingkungan gelap, menawarkan kontras yang sesuai dan mengurangi ketegangan mata. Setiap definisi warna dalam tema gelap memiliki peran yang sama dengan pasangannya di tema terang (misalnya, `md_theme_dark_primary` adalah warna merek utama untuk tema gelap). Penggunaan dua set warna ini memungkinkan aplikasi Anda untuk beralih antara mode terang dan gelap secara dinamis, menyesuaikan tampilan berdasarkan preferensi pengguna atau pengaturan sistem.

28. Themes.xml (light)

Pada baris 1 `<resources>` adalah elemen akar yang menampung berbagai sumber daya dalam aplikasi Android, dan `xmlns:tools` adalah *namespace* untuk atribut *tools* yang membantu pratinjau desain. Baris 3 mendefinisikan sebuah *style* bernama `Base.Theme.ConnectAPI` yang merupakan tema dasar aplikasi. Tema ini mewarisi dari `Theme.Material3.DayNight.NoActionBar`, yang berarti ia mendukung mode terang dan gelap Material Design 3 secara otomatis dan tidak memiliki bilah aksi bawaan. Dari baris 5 hingga 31, terdapat serangkaian `<item>` yang menyesuaikan tema terang aplikasi. Setiap item ini menetapkan atribut tema generik (seperti `colorPrimary`, `colorOnPrimary`, `colorSecondary`, `colorError`, dan lainnya) ke definisi warna spesifik yang telah ditentukan sebelumnya dalam `colors.xml` Anda, seperti `@color/md_theme_light_primary`, `@color/md_theme_light_onPrimary`, dan seterusnya. Ini memastikan bahwa komponen UI Material Design secara otomatis mengadopsi skema warna yang telah Anda tentukan untuk tema terang. Pada

baris 29, `android:statusBarColor` diatur untuk menggunakan warna `colorPrimary` tema Anda, sementara `android:navigationBarColor` pada baris 31 diatur untuk menggunakan `colorSurface` tema Anda, memastikan konsistensi warna di seluruh bilah sistem (status bar dan *navigation bar*). Atribut `tools:targetApi="1"` pada baris 29 dan 31 menunjukkan bahwa pengaturan ini hanya berlaku untuk API level 21 (Lollipop) ke atas.

Terakhir, pada baris 34, `style name="Theme.ConnectAPI"` didefinisikan sebagai tema utama aplikasi Anda, yang cukup mewarisi semua properti dari `Base.Theme.ConnectAPI` yang telah ditentukan sebelumnya. Ini adalah pendekatan umum untuk memisahkan tema dasar yang dapat dimodifikasi dari tema utama yang digunakan oleh aplikasi, memberikan fleksibilitas dalam pengelolaan tema.

29. Themes.xml (night)

Pada baris 1 `<resources>` adalah elemen akar yang menampung berbagai sumber daya dalam aplikasi Android, dan `xmlns:tools` adalah *namespace* untuk atribut `tools` yang membantu pratinjau desain. Baris 3 mendefinisikan sebuah *style* bernama `Base.Theme.ConnectAPI` yang merupakan tema dasar aplikasi. Tema ini mewarisi dari `Theme.Material3.DayNight.NoActionBar`, yang berarti ia mendukung mode terang dan gelap Material Design 3 secara otomatis dan tidak memiliki bilah aksi bawaan.

Dari baris 5 hingga 30, terdapat serangkaian `<item>` yang menyesuaikan tema gelap aplikasi Anda. Setiap `item` ini menetapkan atribut tema generik (seperti `colorPrimary`, `colorOnPrimary`, `colorSecondary`, `colorError`, dan lainnya) ke definisi warna spesifik yang telah ditentukan sebelumnya dalam `colors.xml` Anda, seperti `@color/md_theme_dark_primary`, `@color/md_theme_dark_onPrimary`, dan seterusnya. Ini memastikan bahwa komponen UI Material Design secara otomatis

mengadopsi skema warna yang telah Anda tentukan untuk tema gelap. Pada baris 31, `android:statusBarColor` diatur untuk menggunakan warna `colorPrimary` tema Anda, sementara `android:navigationBarColor` pada baris 33 diatur untuk menggunakan `colorSurface` tema Anda, memastikan konsistensi warna di seluruh bilah sistem (*status bar* dan *navigation bar*). Atribut `tools:targetApi="1"` pada baris 31 dan 33 menunjukkan bahwa pengaturan ini hanya berlaku untuk API level 21 (Lollipop) ke atas.

30. Build.gradle.kts

Pada baris 1-6, blok `plugins` mendeklarasikan *plugin* Gradle yang diterapkan pada modul ini. Ini termasuk `alias(libs.plugins.android.application)` untuk fungsionalitas aplikasi Android dasar, `alias(libs.plugins.kotlin.android)` untuk dukungan Kotlin, `id("kotlin-parcelize")` untuk otomatisasi implementasi `Parcelable` di Kotlin, `id("org.jetbrains.kotlin.plugin.serialization")` versi 2.0.0 untuk dukungan serialisasi Kotlin, dan `id("kotlin-kapt")` untuk pemrosesan anotasi Kotlin. Pada baris 8-39, blok `android` berisi konfigurasi spesifik untuk proyek Android Anda. Baris 9 `namespace = "com.example.connectapi"` mengatur *namespace* Gradle, sedangkan `compileSdk = 35` pada baris 10 menentukan versi SDK yang digunakan untuk kompilasi. Dalam blok `defaultConfig` pada baris 12-20, `applicationId = "com.example.connectapi"` adalah ID unik aplikasi, `minSdk = 30` adalah SDK Android minimum yang didukung, `targetSdk = 35` adalah SDK Android target, `versionCode = 1` dan `versionName = "1.0"` adalah informasi versi aplikasi. `testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"` mengatur

runner untuk pengujian. Pentingnya, `buildConfigField("String", "TMDB_API_KEY", "\"c85206126e56c036e856104caa98a28a\"")` pada baris 19-20 menyuntikkan kunci API TMDB sebagai variabel *build config* sehingga dapat diakses dengan aman dalam kode. Blok `buildTypes` pada baris 22-28 mendefinisikan konfigurasi untuk *build type* yang berbeda. Untuk *release build type*, `isMinifyEnabled = false` menonaktifkan *code minification*, dan `proguardFiles` menentukan aturan ProGuard untuk mengoptimalkan dan mengaburkan kode. Blok `compileOptions` pada baris 30-33 mengatur kompatibilitas *source* dan *target* Java ke `VERSION_11`. Blok `kotlinOptions` pada baris 35-37 mengatur `jvmTarget` untuk Kotlin ke 11. Terakhir, blok `buildFeatures` pada baris 39-42 mengaktifkan `viewBinding` dan `buildConfig`, yang mempermudah interaksi dengan tampilan dan akses ke variabel *build config* yang dibuat otomatis. Pada baris 44-67, blok `dependencies` mendefinisikan semua *library* eksternal yang digunakan proyek Anda. Ini mencakup *library* inti Android seperti `androidx.core.ktx`, `androidx.appcompat`, `material`, `androidx.activity`, dan `androidx.constraintlayout`. Untuk navigasi, ada `androidx.navigation:navigation-fragment-ktx` dan `androidx.navigation:navigation-ui-ktx`. Untuk pemrosesan gambar, `io.coil-kt:coil` digunakan. Untuk pengujian, `junit`, `androidx.junit`, dan `androidx.espresso.core` disertakan. Bagian penting lainnya adalah dependensi terkait basis data dan jaringan: `androidx.room:room-runtime`, `kapt("androidx.room:room-compiler")` untuk pemrosesan anotasi Room, dan `androidx.room:room-ktx` untuk dukungan Kotlin Coroutines dengan Room. Untuk jaringan, ada `com.squareup.retrofit2:retrofit`, `com.squareup.okhttp3:okhttp`, dan `com.squareup.okhttp3:logging-interceptor` (untuk *logging*

permintaan/respons HTTP). Untuk serialisasi JSON dengan Kotlin, `org.jetbrains.kotlin:kotlinx-serialization-json` dan `com.jakewharton.retrofit:retrofit2-kotlinx-serialization-converter` digunakan. Akhirnya, untuk dukungan *coroutine* dan komponen arsitektur Android, ada `org.jetbrains.kotlin:kotlinx-coroutines-android`, `androidx.lifecycle:lifecycle-viewmodel-ktx`, `androidx.lifecycle:lifecycle-runtime-ktx`, dan `androidx.lifecycle:lifecycle-livedata-ktx`.

31. AndroidManifest.xml

Pada file ini, Anda menambahkan baris `<uses-permission android:name="android.permission.INTERNET"/>` yang merupakan deklarasi izin (*permission*) untuk memberi tahu sistem Android bahwa aplikasi membutuhkan akses ke jaringan internet. Izin ini sangat penting terutama jika aplikasi memuat konten dari sumber eksternal seperti gambar dari URL menggunakan *library* Glide. Selain itu, Anda juga menyertakan deklarasi `<activity android:name="com.example.viewmodel.MainActivity">` yang dilengkapi dengan `<intent-filter>` untuk menetapkan MainActivity sebagai titik masuk utama aplikasi. Baris `<action android:name="android.intent.action.MAIN" />` menandakan bahwa aktivitas ini akan dijalankan pertama kali saat aplikasi dibuka, sedangkan `<category android:name="android.intent.category.LAUNCHER" />` memastikan bahwa aktivitas ini akan muncul di daftar aplikasi (*launcher*) pada perangkat Android, sehingga pengguna dapat meluncurkan aplikasi melalui ikon di layar utama. Atribut tambahan seperti `android:theme`, `android:icon`, dan `android:label` digunakan untuk menentukan tampilan dan nama aplikasi di perangkat. Sementara itu, `android:allowBackup`, `android:fullBackupContent`, dan `android:dataExtractionRules` mengatur perilaku pencadangan data

aplikasi, dan `android:supportsRtl="true"` memungkinkan aplikasi mendukung tata letak kanan ke kiri (RTL) untuk bahasa tertentu.

D. Tautan Git

https://github.com/dinzzti/Pemrograman_Mobile